JAVA SDE-2 INTERVIEW PREPARATION SERIES

STUDY STEP 18B OF 18 - PART B OF J

Advanced Java B: Language, OOP, and Modern Java

Objects, Inheritance, Interfaces, Equality, Exceptions, Generics, Lambdas, and Java 21

FOUNDING AUTHOR

Vinay Reddy Kalluri

EDITOR-IN-CHIEF | CHIEF AUDITOR

Develop precise Java language judgment across object design, type systems, equality, exceptions, generics, functional APIs, and modern platform features.

OOP | EQUALITY | EXCEPTIONS | GENERICS | LAMBDAS | JAVA 21

JAVA 21 | INTERVIEW CORE | SDE-2 FOLLOW-UPS | PRINTABLE EDITION

Series edition - Release 2026-07-25

Open educational interview-preparation edition

Start Here - Your Route Through This Volume

> **60-second entry rule:** If two or more recognition signals below expose a gap, begin with Chapter 1. If the prerequisites are weak, step back to `Study Step 18A – Advanced Java A – JVM`. If you can already explain every outcome without notes, jump to Part II and prove it through the practice ladder.

Part B covers the language-engineering depth expected when SDE-2 interviews move beyond coding patterns into API design and type safety.

Readiness map

Decision	Evidence
START HERE	The question tests substitutability or composition; Equality, immutability, or record semantics matter; Generics or reflection expose type-system boundaries; A modern Java feature changes an API design.
PREPARE FIRST	Study Step 01 Java foundations; Study Step 18A execution model recommended.
MOVE ON WHEN	Design cohesive classes and interfaces; Implement equality and immutability correctly; Use exceptions, generics, lambdas, and reflection deliberately; Explain final Java 17 and 21 features accurately.

Choose the route that fits today

1. **Learning:** read in order, type the representative Java examples, and complete each dry run.
2. **Revision or rehearsal:** start at the first decision map, invariant, or failure mode you cannot explain; if none expose a gap, go directly to Part II and prove readiness without notes.

Study Path Position

18A - Advanced Java A - JVM	**YOU ARE HERE** **18B - Advanced Java B - Language**	18C - Advanced Java C - Libraries

Contents

This contents page covers only the current focused PDF. Section-level navigation is available through PDF bookmarks.

CONTENTS NAVIGATION

Part I - Core Learning

SDE-2 CORE CHAPTER

Chapter 1 - Classes, Objects, Access Control, and Packages

Learning objectives

By the end of this chapter, you should be able to:

- separate class definitions, objects, references, identity, and state;
- predict field, initializer, and constructor execution order;
- apply `public`, `protected`, package, and `private` access precisely;
- use static and instance members without confusing ownership; and
- design cohesive classes and package boundaries that preserve invariants.

WHY IT WORKS | Why this matters at SDE-2

Backend design lives in classes and package boundaries. Weak encapsulation lets invalid states spread, couples services to implementation details, and makes concurrent or evolutionary changes dangerous. Strong modeling converts business rules into construction and method contracts.

Java access control also contains interview-worthy subtleties, especially `protected` across packages and the difference between class visibility and member visibility. Experienced engineers should reason about these rules rather than repeatedly widening access until code compiles.

WHY IT WORKS | First-principles model

A class declares a reference type and defines members: fields, methods, constructors, initializers, and nested types. An object is a runtime class instance with identity and field state. A reference value can designate that object or be `null`. Multiple references may designate one object.

Instance members belong conceptually to each object; static members belong to the class's shared namespace and initialization lifecycle. Encapsulation means an object controls how its valid state is created and changed. Access modifiers are compiler- and runtime-enforced visibility rules, not a complete security boundary.

A package groups types and contributes to access control and naming. `com.example.api` and `com.example.api.internal` are distinct packages; package names do not create inheritance of access.

> **Specification boundary:** The Java language specifies initialization order and access checks. It does not promise a field's physical offset, an object's header format, or one object allocation per `new` expression after optimization, provided observable behavior is preserved.

Core terminology

- **Class:** declaration of a reference type and its members.
- **Object:** runtime instance of a class.
- **Identity:** distinction between object instances even when their content is equal.
- **State:** values reachable through an object's instance fields.
- **Invariant:** condition that valid instances maintain.
- **Constructor:** special declaration that initializes a newly allocated instance.
- **Initializer:** static or instance block executed during class or object initialization.
- **Receiver:** object denoted by `this` for an instance method call.
- **Package-private:** access when no modifier is written.
- **Qualified name:** package plus type name, such as `java.time.Instant`.

Detailed mechanics

Fields, methods, and receivers

JAVA

```java
package example.account;

public final class Counter {
    private static int instances;
    private int value;

    public Counter(int initialValue) {
        this.value = initialValue;
        instances++;
    }

    public int increment() {
        return ++value;
    }

    public static int instancesCreated() {
        return instances;
    }
}
```

Each `Counter` has a separate `value`, while all calls observe one static `instances` field per defining class loader. `this` is the current receiver and is unavailable in a static context. A static method should be called

through the class name; calling it through an expression is legal in some forms but misleading because dispatch is not based on that object's runtime class.

The example's shared count is not thread-safe. `++` is a read-modify-write sequence, and `private` says nothing about concurrency.

Construction and initialization order

Object creation conceptually allocates storage, initializes all instance fields to defaults, invokes a selected constructor, and returns a reference if construction completes. Before a constructor body runs, it invokes another constructor in the same class with `this(...)` or a superclass constructor with `super(...)`; if neither is explicit, the compiler inserts an accessible no-argument `super()`.

For each class in the hierarchy, instance field initializers and instance initializer blocks run in textual order after the superclass portion and before that class's constructor body.

JAVA

```java
class Base {
    Base() { System.out.println("Base body"); }
}

class Report extends Base {
    private final String title = initializeTitle();
    { System.out.println("Report initializer"); }

    Report() {
        System.out.println("Report body");
    }

    private String initializeTitle() {
        System.out.println("title initializer");
        return "daily";
    }
}
```

Creating `Report` prints Base body, title initializer, Report initializer, then Report body. Calling an overridable instance method from a constructor is dangerous: dynamic dispatch can reach subclass code before subclass fields have been initialized.

Constructors have no return type and are not inherited. If a class declares no constructor, the compiler supplies a default constructor whose accessibility matches the class. Once any constructor is declared, no default is generated.

Static initialization

Static fields first receive defaults, then static field initializers and static initializer blocks execute in textual order when the class is initialized. Initialization occurs at most once per class or interface per defining class loader, synchronized by the JVM. A failure can leave the class erroneous for subsequent use.

Compile-time constant static fields may be inlined into clients and accessed without triggering initialization of the declaring class. This matters for side effects and binary evolution.

Access control

At top level, a class may be `public` or package-private. A public top-level class is conventionally, and under ordinary file-based compilation, stored in a same-named source file.

Member access proceeds from narrowest to broadest:

- `private`: within the top-level nest that declares it, including its nested nestmates under language access rules;
- package-private: within the same package;
- `protected`: same-package access plus subclass access under an additional cross-package receiver rule;
- `public`: wherever the declaring type and module/package exposure permit.

The cross-package `protected` rule is commonly misunderstood. Subclass code may access the inherited protected member through `this`, `super`, or a reference whose qualifying type is the subclass or its subclass, not through an arbitrary base-class instance.

JAVA

```java
// package library;
public class Base { protected int code; }

// package app;
class Derived extends library.Base {
    void ok(Derived other) { other.code = 1; }
    // void bad(library.Base other) { other.code = 1; }
}
```

`private` members are not inherited as directly accessible members, although an object still contains superclass state and superclass methods can use it. Modern Java nest-based access allows nested classes in the same nest to access one another's private members according to language rules without treating private as public.

Packages and imports

A package declaration must precede imports and type declarations, aside from comments and whitespace. Imports shorten names at compile time; they do not load classes or add runtime dependencies by themselves. `java.lang` is implicitly imported, as are types in the current package. Wildcard imports import types from one package, not subpackages, and do not harm runtime performance.

Static imports can shorten constants or utility calls but may obscure origin. Avoid the unnamed package for production code because named-package code cannot import from it and tooling conventions assume named packages.

Java Platform Module System exports can further restrict whether a public package is accessible to another module. Reflection may also require an `opens` directive; public at the language level is not necessarily globally reflectable in a modular runtime.

TRACE IT | Worked Java example

This class establishes a nonnegative balance invariant and returns new immutable values instead of exposing mutation.

JAVA

```java
package example.money;

import java.util.Objects;

public final class Wallet {
    private final String ownerId;
    private final long cents;

    private Wallet(String ownerId, long cents) {
        this.ownerId = Objects.requireNonNull(ownerId, "ownerId");
        if (ownerId.isBlank()) {
            throw new IllegalArgumentException("blank ownerId");
        }
        if (cents < 0) {
            throw new IllegalArgumentException("negative balance");
        }
        this.cents = cents;
    }

    public static Wallet empty(String ownerId) {
        return new Wallet(ownerId, 0);
    }

    public Wallet deposit(long amountCents) {
        if (amountCents <= 0) {
            throw new IllegalArgumentException("deposit must be positive");
        }
        return new Wallet(ownerId, Math.addExact(cents, amountCents));
    }

    public long balanceCents() {
        return cents;
    }
}
```

The constructor is private so all construction flows remain under the class's control. A factory names the initial state. Final fields plus immutable field values make each instance safe to share after proper construction.

TRACE IT | Execution or memory walkthrough

`Wallet.empty("u-7")` invokes a static method; no receiver object is required. `new Wallet` allocates a new instance whose reference fields initially hold null and whose `long` field holds zero. The constructor validates the copied `ownerId` reference, then assigns both final fields. It returns normally and the factory returns the reference.

Calling `wallet.deposit(500)` supplies the original wallet as receiver and copies `500L` into the parameter. The existing object is not modified. `addExact` computes the new balance and construction validates a new wallet. The caller may retain both versions without aliasing mutable state.

If validation throws, no reference to the incompletely constructed object is returned by the expression. Publishing `this` from a constructor, however, could allow another component to observe incomplete state and must be avoided.

Complexity and performance

Field access and ordinary method dispatch are O(1), excluding method work. Object construction is generally proportional to initialization and reachable defensive copies, not merely the number of fields. The wallet operations are O(1) time and allocate one new object per successful state change.

Immutability can increase allocation but enables safe sharing, caching, and simpler concurrency. A JIT may eliminate non-escaping allocations. Do not replace clear value semantics with mutable pooling unless profiling demonstrates a meaningful bottleneck and ownership remains tractable.

> **HotSpot note:** HotSpot object headers, compressed references, field layout, escape analysis, and allocation in thread-local buffers are implementation choices. Use measurement tools when physical footprint matters; do not derive it only from field widths.

WATCH OUT | Edge cases and common mistakes

- A reference declaration does not create an object.
- `final` on a reference prevents reassignment, not mutation of the object.
- The generated default constructor disappears after any explicit constructor is declared.
- A superclass must be initialized before subclass field initializers run.
- Overridable calls from constructors can observe default-valued subclass fields.
- Leaking `this` during construction can violate invariants and safe publication.
- `private` does not imply thread-safe, immutable, or inaccessible through all reflection configurations.
- Package hierarchy is naming convention, not access inheritance.
- `protected` is not simply package-or-world-visible-to-subclasses; the cross-package qualifying reference matters.
- A public member is unusable if its declaring type is inaccessible.
- Static mutable state is global per class loader, complicates tests, and needs concurrency control.
- Static initialization cycles can expose default values and produce fragile startup failures.

Production engineering notes

Organize packages around stable domain capabilities, not arbitrary technical buckets alone. Expose a small public surface and keep implementation types package-private. If tests need broad access, prefer testing through behavior or narrowly scoped test support over making internals public.

Construct valid objects atomically. Validate required state, use factories or builders when construction has meaningful variants, and avoid setters that create transiently invalid combinations. Keep dependency injection and serialization frameworks from bypassing invariants without explicit adapters.

Treat static state as process-wide infrastructure. Make it immutable where possible; otherwise define lifecycle, synchronization, reset behavior, and class-loader implications. Static constants that may change across versions should not be compile-time constants if clients must observe updates without recompilation.

TRY IT | Interview questions and model answers

What is the difference between a class, an object, and a reference?

A class declares a type and behavior. An object is a runtime instance with identity and state. A reference is a value that designates an object or is null; several references can designate one object.

What is the object initialization order?

Fields receive defaults first. Constructor chaining initializes the superclass portion. For the current class, instance field initializers and instance initializer blocks run in textual order, then its constructor body runs. This repeats down the hierarchy.

How does protected access work across packages?

A subclass may access the protected member as part of its inherited view, but through a qualifying expression whose type is that subclass or a subtype, not an arbitrary instance of the superclass. Same-package code has ordinary package access to it.

Is a private constructor only for singletons?

No. It can enforce factories, canonical instances, validated creation, utility classes, or controlled subclassing. A singleton also needs lifecycle, concurrency, serialization, and testability decisions.

Does importing a type load it?

No. An import is a compile-time name-resolution convenience. Loading and initialization follow runtime use rules, not the presence of an import.

TRY IT | Exercises

1. Predict the print order for a three-level hierarchy with static fields, instance fields, initializer blocks, and constructor chaining.
2. Build a class that cannot represent an invalid date range and has no setters.
3. Create two packages demonstrating legal and illegal protected access through differently typed references.
4. Refactor a public implementation class into a package-private type behind a public interface.
5. Identify all ways `this` could escape from a constructor through callbacks, collections, or threads.
6. Make the `Counter` instance count thread-safe, then discuss whether the global metric belongs in the domain class.

RECAP | Chapter summary

Classes define types; objects carry identity and state; references designate objects. Construction initializes superclass state before subclass initializers and bodies. Access control combines member modifiers, declaring-type accessibility, packages, subclass rules, and possibly modules. Good class design centralizes invariants, limits public surface, avoids construction leaks, and treats static mutable state as an explicit architectural concern.

TRY IT | Revision checklist

- [ ] I distinguish a class, object, reference, and receiver.
- [ ] I can trace static and instance initialization order.
- [ ] I know when the compiler supplies a default constructor.
- [ ] I apply all four member access levels precisely.
- [ ] I can explain the cross-package protected receiver rule.
- [ ] I know that packages and subpackages have no special access relationship.
- [ ] I avoid overridable calls and `this` escape during construction.
- [ ] I design classes whose constructors and methods preserve invariants.

SDE-2 CORE CHAPTER

Chapter 2 - Inheritance, Polymorphism, and Composition

Learning objectives

By the end of this chapter, you should be able to:

- distinguish subtype polymorphism from code reuse and object containment;
- predict overriding, hiding, field access, casts, and constructor behavior;
- apply substitutability when designing and reviewing hierarchies;
- choose composition or inheritance based on semantic ownership; and
- evolve polymorphic APIs without fragile downcasts or superclass coupling.

WHY IT WORKS | Why this matters at SDE-2

SDE-2 design discussions often reduce to one question: where should variation live? A hierarchy can make variation explicit and open to new behavior, but a weak hierarchy spreads superclass assumptions throughout the system. Composition localizes collaboration and runtime configuration, but excessive forwarding can obscure the domain.

Interviewers expect more than definitions of "is-a" and "has-a." They look for dispatch reasoning, substitutability, constructor hazards, and pragmatic trade-offs in service and library design.

WHY IT WORKS | First-principles model

Class inheritance creates a subtype relationship and allows a subclass object to contain a superclass portion. A variable of a supertype can hold a reference to a subtype object. When an overridable instance method is invoked, the object's runtime class chooses the implementation. This is subtype polymorphism.

Inheritance is also a reuse mechanism, but reuse alone is not sufficient justification. A subtype promises that code written against its supertype remains correct. It must preserve the supertype's stated preconditions, postconditions, invariants, and behavioral expectations.

Composition means one object holds or receives another object and delegates part of its work. It creates collaboration without claiming substitutability. Dependencies can often be replaced, combined, or decorated at runtime.

> **Specification boundary:** Java specifies single class inheritance, multiple interface inheritance, method overriding, and dynamic dispatch. Object layout, virtual method tables, inline caches, and devirtualization are JVM implementation strategies, not Java language guarantees.

Core terminology

- **Superclass/subclass:** class being extended and class that extends it.
- **Subtype:** type whose values can be used where a supertype is expected.
- **Upcast:** safe conversion from subtype reference to supertype.
- **Downcast:** checked conversion from supertype reference to a more specific type.
- **Override:** instance method implementation replacing inherited behavior for dispatch.
- **Hide:** static method or field declaration shadows an inherited same-named member.
- **Dynamic dispatch:** runtime selection of an overriding instance method.
- **Substitutability:** subtype preserves contracts expected through the supertype.
- **Composition:** object delegates to contained or injected collaborators.
- **Fragile base class:** superclass changes unexpectedly affect subclasses coupled to internals.

Detailed mechanics

Extending classes

Every class except `Object` has one direct superclass. If `extends` is omitted, it extends `Object`. Constructors are not inherited, and subclass construction must invoke an accessible superclass constructor first.

Inherited accessible members become part of the subclass's behavior. Private superclass state remains present but can be manipulated only through accessible superclass operations. A class declared `final` cannot be subclassed. A method declared `final` cannot be overridden.

Overriding rules

An overriding method has the same subsignature, a compatible covariant reference return, no more restrictive access, and no broader checked exception declaration. `@Override` asks the compiler to verify intent and should be used consistently.

JAVA

```java
class Message {
    CharSequence render() { return "message"; }
}

class Alert extends Message {
    @Override
    String render() { return "alert"; } // covariant return
}
```

Private methods are not overridden. Static methods are hidden and selected by compile-time type. Fields are hidden rather than polymorphic. Instance method calls dispatch dynamically even when called from a superclass method, except for private, static, constructor, or explicit `super` behavior.

JAVA

```java
class Base {
    String name = "base field";
    static String kind() { return "base static"; }
    String describe() { return "base virtual"; }
}

class Derived extends Base {
    String name = "derived field";
    static String kind() { return "derived static"; }
    @Override String describe() { return "derived virtual"; }
}

Base value = new Derived();
System.out.println(value.name);       // base field
System.out.println(Base.kind());     // base static
System.out.println(value.describe());// derived virtual
```

Avoid same-named fields and static hiding because they invite mistaken dispatch assumptions.

super and construction

`super(...)` invokes a direct superclass constructor and must be the first constructor invocation statement, unless another same-class constructor is invoked with `this(...)`. `super.method()` invokes the superclass implementation directly for the current object; it is not a call on a separate superclass object.

During base construction, virtual calls can dispatch to the subclass before subclass initializers run. The result may observe null or zero state and violates the expectation that methods see a fully formed object. Constructors should call private or final helpers when they need internal decomposition.

Casts and instanceof

An upcast needs no explicit syntax and never fails for a non-null compatible reference. A downcast asks the runtime whether the object is an instance of the target type; otherwise it throws `ClassCastException`. Casting null succeeds and produces null.

JAVA

```java
Object candidate = "java";
if (candidate instanceof String text) {
    System.out.println(text.length());
}
```

Pattern variables are in scope where the compiler proves the match succeeded. A design requiring frequent type tests may be missing a polymorphic operation, although boundary adapters and closed algebraic models legitimately inspect variants.

Substitutability

Suppose a base method accepts any integer and promises a nonnegative result. An override cannot reject negative inputs or return negative results without breaking callers. More generally, a subtype should not strengthen preconditions, weaken postconditions, violate invariants, or change important side effects and timing without the contract allowing it.

The classic rectangle/square modeling problem demonstrates that mathematical subset relationships do not automatically make good mutable object subtypes. If a mutable rectangle allows width and height to change independently, a square cannot preserve both that behavior and its equal-sides invariant.

Composition and delegation

Composition represents roles explicitly:

JAVA

```java
interface DiscountPolicy {
    long discountCents(long subtotalCents);
}

final class Checkout {
    private final DiscountPolicy policy;

    Checkout(DiscountPolicy policy) {
        this.policy = java.util.Objects.requireNonNull(policy);
    }

    long totalCents(long subtotalCents) {
        long discount = policy.discountCents(subtotalCents);
        if (discount < 0 || discount > subtotalCents) {
            throw new IllegalStateException("invalid discount");
        }
        return subtotalCents - discount;
    }
}
```

`Checkout` does not claim to be a discount policy. It owns orchestration and validates the collaborator's result. Different policies can be injected without subclasses inheriting checkout internals.

TRACE IT | Worked Java example

This example combines subtype polymorphism at a narrow interface with composition in the service.

JAVA (continued 1/2)

```java
import java.util.List;
import java.util.Objects;

public class NotificationDemo {
    interface Channel {
        Delivery send(String recipient, String body);
    }

    record Delivery(boolean accepted, String providerId) {}

    static final class EmailChannel implements Channel {
        @Override
        public Delivery send(String recipient, String body) {
            return new Delivery(true, "email-42");
        }
    }

    static final class Notifier {
        private final Channel channel;
```

JAVA (continued 2/2)

```java
        Notifier(Channel channel) {
            this.channel = Objects.requireNonNull(channel);
        }

        Delivery notify(String recipient, List<String> lines) {
            String body = String.join(System.lineSeparator(), lines);
            if (body.isBlank()) {
                throw new IllegalArgumentException("empty body");
            }
            return channel.send(recipient, body);
        }
    }

    public static void main(String[] args) {
        Channel channel = new EmailChannel();
        Notifier notifier = new Notifier(channel);
        System.out.println(notifier.notify("a@example.test", List.of("Hello")));
    }
}
```

`Notifier` depends on the role it needs, not the concrete provider. Its validation and message construction stay independent from channel implementations.

TRACE IT | Execution or memory walkthrough

`new EmailChannel()` creates an object referenced through the `Channel` interface. This upcast loses no object information; it narrows which operations are visible at compile time. `Notifier` stores a copied reference to the same channel object.

During `notify`, `String.join` builds the body. The service validates it, then invokes the interface signature `Channel.send`. Runtime dispatch selects `EmailChannel.send`. That method creates and returns a `Delivery` record. No downcast is required anywhere.

If another implementation is injected, the service bytecode still targets the interface operation. Only runtime dispatch changes. The interface contract must therefore specify acceptance meaning, null behavior, failures, and whether retries are safe.

Complexity and performance

Inheritance and composition do not determine algorithmic complexity. Dynamic dispatch and delegation each add conceptually constant overhead. In the example, joining lines takes O(n) time in total character content and O(n) output space; channel work dominates the remaining cost.

Deep inheritance raises cognitive cost even when runtime cost is small. Each override may require understanding implicit superclass state and hooks. Composition adds objects and calls but makes dependency graphs and test seams explicit.

> **HotSpot note:** HotSpot often inlines interface and virtual calls when profiling shows one or a few receiver classes. It can deoptimize if assumptions change. Do not mark classes final solely for speculative call speed without measurement.

WATCH OUT | Edge cases and common mistakes

- Overloading is not overriding; different parameter types create a new overload.
- Fields and static methods do not dispatch on runtime type.
- A downcast changes the reference's static view, not the object, and may fail.
- `instanceof` returns false for null.
- An override cannot reduce accessibility or broaden checked exceptions.
- Calling overridable methods during construction can reach uninitialized subclass state.
- Extending a concrete collection merely to reuse methods often exposes a larger, misleading contract.
- Inheriting from a class with overridable internal hooks creates fragile coupling to call order.
- A base class with `equals` rules that subclasses cannot preserve can break symmetry or transitivity.
- Composition still needs ownership decisions: whether collaborators are thread-safe, mutable, shared, or lifecycle-managed.
- A decorator must preserve the wrapped contract, including exceptions and identity expectations, not only method names.

Production engineering notes

Use inheritance for a genuine, stable subtype relationship with a documented extension contract. Prefer narrow interfaces for roles and composition for configurable behavior, infrastructure clients, and policies. If subclasses are expected, document which methods are hooks, when they run, and what state is valid.

Keep base classes small and protect invariants with private state and final template methods. Avoid exposing protected mutable fields. A protected operation can offer controlled extension without handing subclasses the representation.

Downcasts at system boundaries should fail with useful diagnostics. Within core domain logic, repeated `instanceof` chains often indicate behavior living in the wrong place. For a fixed set of variants, sealed types and pattern matching may make inspection intentional and exhaustive.

TRY IT | Interview questions and model answers

What is runtime polymorphism in Java?

A call to an overridable instance method is dispatched according to the receiver object's runtime class. The compiler has already selected the method signature using static types; runtime chooses the most specific override of that signature.

Why prefer composition over inheritance?

Composition avoids claiming substitutability, limits coupling to a collaborator's public contract, allows runtime replacement, and does not expose superclass hooks or state. Inheritance remains appropriate when values truly are subtypes and the base is designed for extension.

What is the difference between overriding and hiding?

Instance methods override and dispatch dynamically. Static methods and fields can be hidden; access is determined from the compile-time qualifying type. Same-named fields are separate state.

What makes a subtype substitutable?

It honors the supertype's behavioral contract: it accepts at least the promised inputs, returns results meeting the promised postconditions, maintains invariants, and preserves documented failure and side-effect expectations.

Why are constructor calls to overridable methods unsafe?

Dynamic dispatch reaches the subclass implementation before the subclass's field initializers and constructor body have run. The method can observe default state or leak the incomplete object.

TRY IT | Exercises

1. Predict field, static method, and instance method outputs through base and derived references.
2. Design a subclass that violates a base precondition contract, then refactor it into composition.
3. Replace a three-branch `instanceof` chain with a polymorphic operation.
4. Implement a timing decorator for `Channel` that preserves exceptions and return values.
5. Analyze whether `Stack extends ArrayList` would be substitutable and identify inherited operations that violate stack intent.
6. Write a base constructor that accidentally dispatches to subclass state, demonstrate the failure, and repair it with a private helper.

RECAP | Chapter summary

Inheritance creates a subtype promise, not merely a reuse opportunity. Compile-time member selection and runtime overriding must be considered separately: fields and static methods hide, while overridable instance methods dispatch. Subtypes must preserve contracts. Composition expresses collaboration without exposing superclass internals and is usually safer for configurable behavior. Choose the relationship that tells the domain truth and keeps invariants local.

TRY IT | Revision checklist

- [ ] I distinguish subtype polymorphism from implementation reuse.
- [ ] I can trace overriding, field hiding, and static hiding.
- [ ] I understand upcasts, downcasts, and pattern `instanceof`.
- [ ] I can state substitutability in terms of behavioral contracts.
- [ ] I avoid virtual calls from constructors.
- [ ] I recognize fragile base-class coupling.
- [ ] I choose composition for roles and runtime-configurable policies.
- [ ] I document extension points and collaborator ownership explicitly.

SDE-2 CORE CHAPTER

Chapter 3 - Interfaces, Abstract Classes, Sealed Types, and Pattern Matching

Learning objectives

By the end of this chapter, you should be able to:

- choose among interfaces, abstract classes, sealed hierarchies, and composition;
- resolve default-method inheritance and understand interface member rules;
- design and enforce a closed subtype family with `sealed`, `non-sealed`, and `final`;
- use Java 21 type patterns, record patterns, and pattern switch exhaustively; and
- identify which pattern features were preview versus permanent in Java 17 and 21.

WHY IT WORKS | Why this matters at SDE-2

Backend systems repeatedly model variants: payment results, commands, events, failures, and protocol messages. An open interface supports third-party implementations; a sealed algebra supports a known set of cases. Choosing incorrectly either blocks extension or makes exhaustive reasoning impossible.

Java 21 pattern matching makes closed models concise, but concision is not the primary benefit. The real gain is aligning domain completeness with compiler checks. SDE-2 interviews increasingly test this modern type-system reasoning while still expecting knowledge of default methods and abstract base classes.

WHY IT WORKS | First-principles model

An interface specifies a reference type whose instances are objects of implementing classes. It supports multiple inheritance of type and behavior but not per-instance interface state. An abstract class is a partially implemented class with constructors and instance fields, and a subclass can extend only one class.

A sealed type restricts which classes or interfaces may directly extend or implement it. Each permitted direct subtype must explicitly continue the policy: `final` closes it, `sealed` declares another restricted level, or `non-sealed` reopens it.

A pattern combines a test with conditional variable extraction. Pattern matching does not bypass the type system; it moves a cast and its proof into one construct. Exhaustive pattern switch is especially valuable when the selector is a closed hierarchy.

> **Specification boundary:** Exhaustiveness, dominance, permitted subclasses, and pattern-variable scope are compile-time language rules. The JVM also records permitted subclasses in class-file metadata and rejects unauthorized direct extension; it does not guarantee that a sealed hierarchy has only a fixed number of runtime object instances.

Core terminology

- **Interface:** contract type supporting multiple implementation inheritance.
- **Default method:** interface instance method with an implementation.
- **Abstract class:** non-instantiable class that may mix abstract behavior, state, and implementation.
- **Sealed type:** type with an enumerated set of permitted direct subtypes.
- **Non-sealed type:** permitted subtype that reopens unrestricted inheritance.
- **Pattern:** test that conditionally binds variables from a value.
- **Type pattern:** tests a runtime type and binds the narrowed value.
- **Record pattern:** deconstructs record components recursively.
- **Dominance:** an earlier case makes a later case unreachable.
- **Exhaustiveness:** cases cover every possible selector value allowed by the type rules.

Detailed mechanics

Interface members and evolution

Interface fields are implicitly `public static final` and must have initializers. Interface methods can be:

- implicitly `public abstract` when declared without a body;
- `public default` instance methods with bodies;
- `public static` methods; or
- `private` instance or static helpers, available since Java 9.

Implementations cannot reduce the accessibility of an interface's public method. Static interface methods are called through the interface and are not inherited as instance behavior. An interface has no constructor and cannot hold per-object instance fields.

Default methods allow compatible interface evolution, but conflicts follow rules:

1. a concrete class method wins over an interface default;
2. a more specific subinterface default wins over a less specific one; and
3. unrelated inherited defaults with the same signature require an explicit override.

```java
interface JsonForm { default String format() { return "json"; } }
interface TextForm { default String format() { return "text"; } }

final class Output implements JsonForm, TextForm {
    @Override
    public String format() {
        return JsonForm.super.format();
    }
}
```

`InterfaceName.super.method()` resolves a direct superinterface default. An abstract declaration in a more specific interface can also remove an inherited default and force implementers to provide behavior.

Abstract classes

An abstract class may declare abstract methods and concrete members. It can have constructors even though it cannot be instantiated directly; subclass constructors use them to initialize the base portion. A class containing an abstract method must be abstract.

Use an abstract class when implementations share protected invariants, instance state, or a controlled template algorithm. Keep extension contracts explicit and avoid protected mutable fields. Use an interface when the important concept is a role that unrelated classes can implement or when multiple roles must compose.

```java
abstract class BatchJob {
    public final void run() {
        validate();
        execute();
    }

    protected void validate() {}
    protected abstract void execute();
}
```

The final template fixes sequencing while hooks provide limited variation. It still inherits constructor and fragile-base-class risks, so document whether hooks may throw, block, or mutate shared state.

Sealed hierarchies

Sealed classes and interfaces became permanent in Java 17. A declaration names permitted direct subtypes with `permits`, unless the compiler can infer them from declarations in the same source file.

```java
sealed interface Command permits Create, Cancel {}
record Create(String id) implements Command {}
record Cancel(String id) implements Command {}
```

Records are implicitly final, so they satisfy the continuation requirement. A permitted ordinary class must declare `final`, `sealed`, or `non-sealed`.

Permitted direct subtypes must be accessible and compiled in the same named module as the sealed type. In an unnamed module, they must be in the same package. Sealing controls direct inheritance, not visibility: a permitted subtype can itself expose behavior according to its modifiers.

Sealed types are ideal for closed domain alternatives maintained together. They are a poor public extension point when independent consumers must add implementations. `non-sealed` deliberately creates an open branch, which can reduce switch exhaustiveness over deeper runtime variants even though the direct branch is known.

Type patterns

Pattern matching for `instanceof` became permanent in Java 16 and is therefore permanent in both Java 17 and 21.

JAVA

```java
static int length(Object value) {
    if (value instanceof String text && !text.isEmpty()) {
        return text.length();
    }
    return 0;
}
```

The pattern variable is in scope only where flow analysis proves the match. With &&, it is available on the right. With a negated test followed by an abrupt exit, it can be available afterward:

JAVA

```java
if (!(value instanceof String text)) {
    throw new IllegalArgumentException();
}
System.out.println(text.length());
```

Patterns do not match null. A type pattern that would be unconditionally true from the static type is generally rejected as pointless, while casts have a different compatibility history.

Pattern switch in Java 21

Pattern matching for `switch` was preview in Java 17 under JEP 406 and required `--enable-preview`. It evolved across releases and became permanent in Java 21 under JEP 441. Java 17 preview syntax and semantics should not be treated as a stable Java 21 source contract.

JAVA

```java
static String describe(Object value) {
    return switch (value) {
        case null -> "missing";
        case String text when text.isBlank() -> "blank text";
        case String text -> "text of length " + text.length();
        case Integer number -> "integer " + number;
        default -> "other";
    };
}
```

Java 21 uses `when` guards. Cases are tried in source order subject to compiler dominance rules. `case Object ignored` dominates subtype cases after it and must therefore appear last. A guarded pattern generally does not dominate the same unguarded pattern unless its guard is a constant true expression.

Pattern switch supports explicit `case null`; without a matching null label, switching on null throws `NullPointerException`. Type and record patterns do not match null, so null handling must be explicit when it is part of the domain.

An exhaustive switch expression over a sealed selector can omit `default` when all permitted alternatives are covered. Omitting a broad default is often desirable: when the hierarchy changes and code is recompiled, the compiler points to every incomplete switch. The compiler still emits a defensive failure path for incompatible binary evolution.

Record patterns in Java 21

Record patterns became permanent in Java 21 under JEP 440. They deconstruct records and can nest:

JAVA

```java
record Point(int x, int y) {}
record Segment(Point start, Point end) {}

static int horizontalLength(Object value) {
    return switch (value) {
        case Segment(Point(int x1, int y1), Point(int x2, int y2))
                when y1 == y2 -> Math.abs(x2 - x1);
        default -> 0;
    };
}
```

Record patterns were not part of Java 17. Java 21 also permits `var` in record component patterns where inference is useful. A record pattern checks and extracts component values; accessor invocation and nested matching can still fail only according to ordinary execution and pattern rules, so keep record accessors pure.

TRACE IT | Worked Java example

This closed result type turns success, rejection, and transient failure into explicit cases.

JAVA

```java
public class PaymentResultDemo {
    sealed interface PaymentResult
            permits Approved, Rejected, RetryLater {}

    record Approved(String authorizationId) implements PaymentResult {}
    record Rejected(String reason) implements PaymentResult {}
    record RetryLater(long delayMillis) implements PaymentResult {}

    static String clientMessage(PaymentResult result) {
        return switch (result) {
            case Approved(String id) -> "approved: " + id;
            case Rejected(String reason) -> "rejected: " + reason;
            case RetryLater(long delay) when delay <= 1_000 -> "retry soon";
            case RetryLater(long delay) -> "retry in " + delay + " ms";
        };
    }

    public static void main(String[] args) {
        System.out.println(clientMessage(new Approved("auth-7")));
        System.out.println(clientMessage(new RetryLater(500)));
    }
}
```

This source requires Java 21 because it uses record patterns and a guarded pattern switch. It needs no preview flag in Java 21.

TRACE IT | Execution or memory walkthrough

`new RetryLater(500)` creates a final record instance referenced as `PaymentResult`. `clientMessage` first checks alternatives according to the selector's runtime class. The two unrelated record types do not match. The `RetryLater(long delay)` pattern matches, extracts the primitive component into `delay`, and evaluates the guard.

Because 500 is at most 1000, the guarded arm produces `"retry soon"`; the unguarded RetryLater arm is not evaluated. Exhaustiveness follows from all three direct permitted record implementations, with the two RetryLater arms collectively covering that variant.

If a fourth permitted subtype is added and this code is recompiled, the compiler rejects the incomplete switch. If separately compiled binaries become inconsistent, a generated defensive path prevents silently treating the new value as an old case.

Complexity and performance

Interface calls, virtual calls, type tests, component extraction, and ordinary switch selection are conceptually O(1), excluding method bodies. Nested patterns add work proportional to the number of tested components and guards. Sealed metadata can help tools and optimizers reason about possible subtypes, but its primary benefit is semantic completeness.

An abstract template can avoid repeated orchestration code; an interface-plus-composition design may add delegation. These constant costs are typically irrelevant beside I/O. Optimize the model for comprehensibility and profile before specializing dispatch.

> **HotSpot note:** HotSpot may devirtualize interface calls and optimize type tests using class profiling. Sealing can provide additional hierarchy facts, but no source-level latency or allocation guarantee follows from declaring a type sealed.

WATCH OUT | Edge cases and common mistakes

- Interface fields are static constants, not per-implementation state.
- A class method wins over an interface default, even if inherited from a superclass.
- Unrelated same-signature defaults require an explicit resolution.
- Abstract classes may have constructors; interfaces do not.
- A sealed type's permitted direct subclasses must satisfy module or package placement rules.
- Every permitted direct subtype must be final, sealed, or non-sealed.
- `non-sealed` is a deliberate reopening, not the absence of a decision.
- Patterns do not behave like unchecked destructuring; type compatibility and dominance are enforced.
- A broad case before a narrow case can make the latter dominated and illegal.
- Null handling in pattern switch must be intentional.
- Java 17 pattern switch was preview and changed before finalization; compile Java 21 examples as Java 21 source.
- Record patterns are final in Java 21 but unavailable in Java 17.
- An exhaustive switch without default is often better for closed models, but binary evolution still needs operational compatibility planning.

Production engineering notes

Use open interfaces for provider ecosystems and test seams. Use sealed types for centrally owned outcomes, commands, or syntax trees whose variants must be exhaustively understood. Avoid sealing across teams when deployments and extension ownership are independent.

Keep default methods small and contract-preserving. Adding a default method can still conflict with another interface a consumer already implements. Library evolution requires downstream compatibility testing, not only successful compilation of the library.

Pattern switches should translate or interpret variants at clear architectural boundaries. If every consumer repeats a large switch, behavior may belong on the variants or in a visitor-like service. Conversely, putting every external concern into domain variants can create coupling; exhaustive external interpreters are often appropriate.

TRY IT | Interview questions and model answers

When should you use an interface instead of an abstract class?

Use an interface for a role that unrelated classes can implement, multiple inheritance of type, or an open provider contract. Use an abstract class when closely related implementations share state, construction rules, and a controlled implementation skeleton. Composition can combine either.

How are conflicting default methods resolved?

A class implementation wins over interface defaults. A more specific subinterface wins over its ancestor. Unrelated defaults require the implementing class to override and optionally delegate with `InterfaceName.super.method()`.

What does sealing guarantee?

Only listed permitted types may directly extend or implement the sealed type, subject to module or package rules. Each permitted subtype states whether its own branch closes, stays sealed, or reopens. It does not imply immutability or a fixed object count.

Which pattern features are final in Java 17?

Type patterns for `instanceof` are final, and sealed classes are final features in Java 17. Pattern matching for switch is preview in Java 17. Record patterns are not available there.

Which features became final in Java 21?

Pattern matching for switch and record patterns are permanent in Java 21. Java 21 guarded case labels use `when`. Neither feature requires `--enable-preview` in Java 21.

TRY IT | Exercises

1. Resolve a diamond of two unrelated default methods while calling both implementations deliberately.
2. Model a sealed file-operation result and write an exhaustive Java 21 switch without default.
3. Add a non-sealed branch and explain what remains known to the compiler.
4. Convert nested casts for two records into a nested record pattern.
5. Write a pattern switch whose cases are initially dominated, then reorder or guard them correctly.
6. Decide whether a plugin API should be sealed; document extension ownership, module boundaries, and compatibility consequences.

RECAP | Chapter summary

Interfaces model roles and support multiple inheritance of contract and default behavior. Abstract classes provide single-inheritance state and controlled templates. Sealed types make a directly permitted family explicit, while each branch chooses whether to close or reopen. Java 21's final pattern switch and record patterns let code test, extract, guard, and exhaustively handle those models. Java 17 supports final sealed types and `instanceof` patterns, but its pattern switch is preview.

TRY IT | Revision checklist

- [ ] I can choose between an interface, abstract class, sealed type, and composition.
- [ ] I know interface field, static, default, and private-method rules.
- [ ] I can resolve default-method conflicts.
- [ ] I understand permits placement and final/sealed/non-sealed continuation.
- [ ] I use flow-scoped `instanceof` pattern variables safely.
- [ ] I can order pattern switch cases without dominance errors.
- [ ] I handle null and exhaustiveness deliberately.
- [ ] I can label Java 17 preview and Java 21 permanent pattern features accurately.

SDE-2 CORE CHAPTER

Chapter 4 - Equality, Hashing, Immutability, and Records

Learning objectives

By the end of this chapter, you should be able to:

- distinguish reference identity, logical equality, ordering, and domain identity;
- implement the `equals` and `hashCode` contracts together;
- explain why mutable hash keys and subclass equality are dangerous;
- design deeply immutable value objects with safe publication; and
- use records while understanding their generated members and shallow immutability.

WHY IT WORKS | Why this matters at SDE-2

Equality is infrastructure for collections, caches, deduplication, persistence, tests, and distributed idempotency. A broken contract does not always fail immediately; it can make an entry unreachable in a map or create asymmetric behavior that changes with argument order.

Records reduce value-carrier boilerplate, but they do not choose a domain model for you. An SDE-2 engineer must decide which fields define equality, whether data is deeply immutable, and whether an ORM entity, request DTO, or domain value should be a record at all.

WHY IT WORKS | First-principles model

Every object has identity. Reference `==` asks whether two reference values designate the same object, with null handled as an ordinary reference value. `Object.equals` initially implements identity equality, but classes can override it to define logical value equality.

Hashing compresses equality-relevant state into an integer used to choose candidate buckets. A hash is not a unique identifier. Equal objects must have equal hash codes; unequal objects may collide. Hash-based collections use both hash and equality.

Immutability means an object's observable state cannot change after construction. It requires control of the entire reachable representation, not only final top-level fields. A record is a transparent nominal product of its components with generated accessors and value-oriented members. Its component fields are final, but referenced objects may be mutable.

> **Specification boundary:** Java specifies the `Object.equals` and `hashCode` contracts and record member semantics. It does not specify a `HashMap` bucket layout here, a stable hash code across JVM runs, or that `Object.hashCode` is a memory address.

Core terminology

- **Identity equality:** same runtime object, tested with reference `==`.
- **Logical equality:** class-defined equivalence, tested by `equals`.
- **Domain identity:** business identity, such as an immutable account identifier.
- **Value object:** object defined by its values rather than lifecycle identity.
- **Hash collision:** unequal values have the same hash code.
- **Consistency:** repeated equality or hash calls return compatible results while relevant state is unchanged.
- **Deep immutability:** no observable reachable mutable state can change through any alias.
- **Defensive copy:** independent representation used to prevent outside mutation.
- **Record component:** declared state item that generates a field, accessor, and participation in generated members.
- **Canonical constructor:** record constructor whose parameters correspond to all components.

Detailed mechanics

The equals contract

For non-null references, a correct equivalence relation is:

- reflexive: `x.equals(x)` is true;
- symmetric: `x.equals(y)` equals `y.equals(x)`;
- transitive: if x equals y and y equals z, x equals z;
- consistent while equality-relevant information is unchanged; and
- false for `x.equals(null)`.

An implementation normally checks identity, compatible type, and relevant fields:

JAVA

```java
@Override
public boolean equals(Object other) {
    if (this == other) return true;
    if (!(other instanceof Coordinate that)) return false;
    return x == that.x && y == that.y;
}
```

`getClass()` versus `instanceof` is a design choice. Exact-class equality is easier to keep symmetric in extensible hierarchies but means a base and subclass never compare equal. `instanceof` can support equality across implementations only if every participant shares the same semantics. Making value classes final or records avoids many subclass traps.

The hashCode contract

If `x.equals(y)`, then `x.hashCode() == y.hashCode()` must hold. The reverse is not required. The result must be consistent while equality state is unchanged. Overriding one of `equals` and `hashCode` without the other is almost always a defect.

JAVA

```java
@Override
public int hashCode() {
    int result = Integer.hashCode(x);
    result = 31 * result + Integer.hashCode(y);
    return result;
}
```

`Objects.hash(x, y)` is concise but uses varargs and boxing, which can matter on a hot path. Hand-written accumulation or generated code avoids that allocation. Quality matters because clustered hashes degrade hash-table performance, but correctness still relies on equality.

Never base logical hash solely on a mutable field if objects can be keys. If a key changes after insertion, lookup uses its new hash or equality state while the entry remains located according to its old state.

Equality across common types

Arrays retain identity `equals`; use `Arrays.equals` or `deepEquals`. `BigDecimal.equals` considers value and scale, so `1.0` is not equal to `1.00`; `compareTo` considers them numerically equal. This makes `BigDecimal` behavior differ between hash-based and sorted collections if the chosen semantics are not normalized.

Enums use identity safely because each declared constant is a canonical instance in its defining class loader context, and `Enum.equals` is final. Collections define content equality, usually including iteration order for lists and membership for sets.

Entity equality is difficult when database-generated IDs begin as null and proxies introduce subclasses. Do not mechanically include every mutable field. Choose a stable business key, controlled persistent identity policy, or explicit reference identity based on lifecycle requirements.

Building immutable classes

A typical immutable class is final, has private final fields, validates fully during construction, exposes no mutators, and copies mutable inputs and outputs. Final-field semantics aid safe publication when construction is correct, but final references alone do not freeze mutable referents.

JAVA

```java
public final class Tags {
    private final java.util.List<String> values;

    public Tags(java.util.Collection<String> values) {
        this.values = java.util.List.copyOf(values);
    }

    public java.util.List<String> values() {
        return values;
    }
}
```

`List.copyOf` rejects nulls and returns an unmodifiable snapshot when needed. If elements are mutable, copy or transform them too for deep immutability. Unmodifiable wrappers alone are views: another alias may still mutate the backing collection.

Avoid letting `this` escape during construction. Safe final-field visibility assumes the object is not observed before constructor completion.

Records

Records became permanent in Java 16 and are available in Java 17 and 21. A declaration such as:

JAVA

```java
record Coordinate(int x, int y) {}
```

implicitly defines a final class extending `java.lang.Record`, private final component fields, public component accessors `x()` and `y()`, a canonical constructor, and generated `equals`, `hashCode`, and `toString`. It can implement interfaces and declare static or instance methods, static fields, and additional constructors, but cannot declare extra instance fields or extend another class.

Generated record equality uses the same record class and component values. Its exact hash combination should not be treated as a stable serialization or partitioning algorithm. Record `toString` is convenient for diagnostics but can expose sensitive components if logged.

A compact canonical constructor validates or normalizes components. Assignments to component fields occur implicitly after its body using the possibly reassigned parameter values.

JAVA

```java
record EmailAddress(String value) {
    EmailAddress {
        value = java.util.Objects.requireNonNull(value).strip();
        if (!value.contains("@")) {
            throw new IllegalArgumentException("invalid email");
        }
    }
}
```

Explicit canonical constructors cannot reduce accessibility below the record's accessibility. Records are not automatically deeply immutable, and their accessors expose component references directly.

TRACE IT | Worked Java example

This record normalizes permission spelling and takes an immutable snapshot, producing stable equality and hashing.

JAVA

```java
import java.util.Set;
import java.util.TreeSet;

public record PermissionSet(String principalId, Set<String> permissions) {
    public PermissionSet {
        if (principalId == null || principalId.isBlank()) {
            throw new IllegalArgumentException("invalid principalId");
        }
        if (permissions == null) {
            throw new IllegalArgumentException("permissions is null");
        }
        TreeSet<String> normalized = new TreeSet<>();
        for (String permission : permissions) {
            if (permission == null || permission.isBlank()) {
                throw new IllegalArgumentException("invalid permission");
            }
            normalized.add(permission.strip().toLowerCase(java.util.Locale.ROOT));
        }
        permissions = Set.copyOf(normalized);
    }

    public boolean allows(String permission) {
        return permissions.contains(permission.toLowerCase(java.util.Locale.ROOT));
    }

    public static void main(String[] args) {
        PermissionSet a = new PermissionSet("u1", Set.of("READ", "write"));
        PermissionSet b = new PermissionSet("u1", Set.of("write", "read"));
        System.out.println(a.equals(b)); // true
        System.out.println(a.allows("READ")); // true
    }
}
```

Because set equality is order-independent and the canonical constructor normalizes content, different input order and case lead to the same value. Whether permission identifiers should be case-insensitive is a domain decision, not a universal rule.

TRACE IT | Execution or memory walkthrough

Construction receives copied references to the principal string and caller set. It validates the identifier, creates a new `TreeSet`, normalizes each permission, and then snapshots that set using `Set.copyOf`. Reassigning the constructor parameter changes the value that the compiler assigns to the final component field.

The caller's original set is not retained. The record is shallowly immutable by structure, and this particular representation is deeply immutable because strings are immutable and the set cannot be changed. Generated `equals` compares the same record type and then corresponding component values. Both instances contain equal strings and equal sets, so their hash codes must also match.

`allows` performs normalization without mutating the record. Passing null currently throws `NullPointerException`; a public API should document that or validate with a clearer message.

Complexity and performance

`equals` is proportional to compared state until a difference is found. Hash computation is similarly proportional unless a safely cached hash is used. Hash-table lookup is expected O(1) with well-distributed hashes and controlled load, but collisions and adversarial inputs can change costs.

Immutable values enable sharing and memoization but defensive copies cost O(n) time and space. `PermissionSet` normalization is O(n log n) because of `TreeSet`; if sorted construction is unnecessary, a hash set can offer expected O(n). The final `Set.copyOf` may add another pass.

> **HotSpot note:** Generated record methods are ordinary methods that the JVM may inline. Object allocation or field reads may be optimized, but records are still reference objects and do not imply stack allocation or flattened storage.

WATCH OUT | Edge cases and common mistakes

- `==` on references tests identity, not logical value.
- Equal objects must share a hash, but equal hashes do not imply equal objects.
- Mutable map keys can become unreachable after insertion.
- `getClass` and `instanceof` strategies can break symmetry when mixed across a hierarchy.
- Including an array field with `Objects.equals` compares array identity; use the matching `Arrays` helper.
- `BigDecimal.equals` includes scale while `compareTo` does not.
- Unmodifiable collection wrappers do not copy their backing collections.
- Final fields do not make mutable elements immutable.
- Records can contain arrays, lists, or date objects that mutate.
- A compact record constructor can reassign parameters for normalization but should not leak partially constructed state.
- Generated `toString` can leak credentials or personal data.
- Hash codes are not stable persistence keys, signatures, or security hashes.

Production engineering notes

Write equality from domain identity, not from whatever fields happen to exist today. Value objects usually include all normalized value components. Entities need a lifecycle-aware identity policy that behaves before and after persistence and with framework proxies.

If a type is used as a cache or map key, make equality-relevant state immutable. Add contract tests for reflexivity, symmetry, transitivity, equal hashes, null, and collection behavior. Property-based tests can explore combinations better than a few examples.

Use records for transparent carriers when exposing components matches the API. Prefer a conventional class when representation must stay hidden, construction requires staged mutable machinery, or framework constraints conflict. Redact sensitive values explicitly rather than trusting generated output.

TRY IT | Interview questions and model answers

What is the relationship between equals and hashCode?

If two objects are equal, their hash codes must be equal. Unequal objects may share a hash. Both results must stay consistent while equality state is unchanged. Therefore a class that overrides logical equality must provide a compatible hash implementation.

Why is a mutable HashMap key dangerous?

The map places it according to its hash at insertion. If equality-relevant state changes, lookup computes a different hash or comparison and may not find the existing entry. Immutability is the simplest key policy.

Are records immutable?

Their component fields are final and the record class is final, so component references cannot be reassigned after construction. The objects referenced by components can still mutate. Defensive copies are required for deep immutability.

Should entity equality include every field?

Usually not. Mutable fields make hashing unstable and two lifecycle instances may represent the same entity. Choose a stable business identity or a carefully specified persistent-identity strategy that handles transient instances and proxies.

Can a record extend a base class?

No. Every record implicitly extends `java.lang.Record` and is final. It may implement interfaces and define behavior.

TRY IT | Exercises

1. Implement a final `Money` value with currency and minor units, including contract tests.
2. Demonstrate a map lookup failure after mutating a key, then repair the key design.
3. Compare `BigDecimal("1.0")` and `BigDecimal("1.00")` in `HashSet` and `TreeSet`; explain the result.
4. Make a record with a `byte[]` component deeply immutable, including accessor behavior.
5. Construct a base/subclass equality implementation that breaks symmetry, then redesign it.
6. Decide whether a JPA entity, API DTO, and domain identifier should each be a record, with reasons.

RECAP | Chapter summary

Identity and equality answer different questions. Logical equality must be an equivalence relation, and equal values must have equal hashes. Mutable equality state is unsafe for hash keys. Immutability requires control of reachable mutable data and careful publication. Records generate transparent value-oriented structure but remain shallowly immutable reference objects; domain normalization, validation, secrecy, and equality meaning are still design responsibilities.

TRY IT | Revision checklist

- [ ] I can state every equals contract property.
- [ ] I implement equals and hashCode together from the same state.
- [ ] I do not treat hashes as unique or stable identifiers.
- [ ] I avoid mutable equality state in collection keys.
- [ ] I can explain exact-class versus cross-class equality trade-offs.
- [ ] I distinguish final fields, unmodifiable views, and deep immutability.
- [ ] I know what members records generate and what they do not guarantee.
- [ ] I validate, normalize, copy, and redact record components as required.

SDE-2 CORE CHAPTER

Chapter 5 - Exceptions and Resource Management

Learning objectives

By the end of this chapter, you should be able to:

- distinguish checked exceptions, unchecked exceptions, and errors by contract;
- trace matching, propagation, rethrow, `finally`, and suppressed exceptions;
- use try-with-resources with correct ownership and close order;
- translate failures without losing cause or actionable context; and
- design retryable, observable, and non-leaking production error boundaries.

WHY IT WORKS | Why this matters at SDE-2

Failure handling determines whether a service preserves data and recovers predictably. A lost cause obscures incidents, an overbroad catch turns corruption into a fake success, and a leaked connection can exhaust a pool. SDE-2 engineers must design failure semantics, not merely make the compiler accept `throws` clauses.

Interviews commonly ask about checked versus unchecked exceptions, `finally` behavior, close ordering, and what happens when both work and cleanup fail. The strongest answers connect the mechanics to API ownership and operational policy.

WHY IT WORKS | First-principles model

An exception is an object representing abrupt completion. `throw` transfers control up dynamically nested execution until a compatible `catch` is found. If no application frame catches it, the thread terminates after its uncaught-exception handling.

Java divides `Throwable` into `Error` and `Exception`. `RuntimeException` and its subclasses are unchecked, as are `Error` subclasses. Other `Exception` subclasses are checked: a potentially escaping checked exception must be caught or declared.

A resource is something with a lifecycle that must be closed regardless of normal or abrupt completion. Try-with-resources gives that lifecycle structured scope. Ownership answers who closes; it is separate from which variable can access the resource.

> **Specification boundary:** Java specifies exception search, `finally`, try-with-resources translation, close order, and suppression. It does not guarantee that a process can recover from every `Error`, that every termination path runs cleanup, or that stack-trace collection has a fixed cost.

Core terminology

- **Checked exception:** non-`RuntimeException` subclass of `Exception`, enforced by compile-time handling rules.
- **Unchecked exception:** `RuntimeException`, `Error`, or subclass.
- **Cause:** lower-level failure wrapped by another exception.
- **Stack trace:** recorded execution frames associated with a throwable.
- **Suppressed exception:** secondary failure retained when another failure is primary.
- **Try-with-resources:** statement that closes `AutoCloseable` resources automatically.
- **Exception translation:** mapping an implementation failure to a boundary-appropriate abstraction.
- **Precise rethrow:** compiler infers narrower exceptions when rethrowing an effectively final catch parameter.
- **Idempotency:** repeated operation has an effect compatible with safe retry.
- **Ownership:** responsibility for closing or releasing a resource.

Detailed mechanics

Throwing and declaring

`throw` requires a `Throwable` reference; throwing null causes `NullPointerException`. A `throws` clause declares possible checked failures for callers but does not itself throw anything. Unchecked failures may be documented without being declared.

Checked exceptions can communicate a recoverable condition a caller is expected to consider. Unchecked exceptions commonly represent violated preconditions, programming defects, or failures handled at a wider boundary. This is an API design choice, not a guarantee that checked failures are recoverable or unchecked failures are fatal.

JAVA

```java
static int parsePort(String text) {
    int port;
    try {
        port = Integer.parseInt(text);
    } catch (NumberFormatException ex) {
        throw new IllegalArgumentException("port is not numeric: " + text, ex);
    }
    if (port < 1 || port > 65_535) {
        throw new IllegalArgumentException("port out of range: " + port);
    }
    return port;
}
```

Translation keeps the original exception as cause and adds domain context. Avoid putting secrets in messages.

Catch selection and propagation

Catch clauses are tested in source order, and the first assignment-compatible type handles the throwable. Therefore a subtype catch must precede a supertype catch or the latter makes it unreachable. After the catch runs normally, execution continues after the whole try statement.

Multi-catch uses alternatives such as `catch (IOException | SQLException ex)`. Alternatives cannot be related by subclassing, and the catch parameter is implicitly final because assigning it would make its type unclear.

Precise rethrow lets this pattern declare the specific possible checked types rather than broad `Exception` when the caught parameter is not reassigned:

JAVA

```java
static void invoke() throws java.io.IOException, java.sql.SQLException {
    try {
        callThatThrowsEither();
    } catch (Exception ex) {
        audit(ex);
        throw ex;
    }
}
```

This should not justify catching broad types routinely; it is useful at cross-cutting boundaries that must observe and rethrow unchanged.

Finally and abrupt completion

A `finally` block executes after its associated try/catch completes normally or abruptly, before control continues outward. If `finally` itself returns or throws, it replaces the pending return value or exception. That can silently lose the true failure.

JAVA

```java
static int misleading() {
    try {
        return 1;
    } finally {
        return 2; // replaces the pending return; avoid this
    }
}
```

Cleanup may not run if the process halts, is externally killed, the JVM crashes, or execution cannot proceed. Resource safety is scoped, not a substitute for durable recovery protocols.

Try-with-resources

A resource type implements `AutoCloseable`; `Closeable` is a narrower I/O-oriented subtype whose `close` declares `IOException`. Resources initialize left to right and close in reverse order.

JAVA

```java
try (var input = java.nio.file.Files.newBufferedReader(path);
     var output = java.nio.file.Files.newBufferedWriter(target)) {
    output.write(input.readLine());
}
```

If initialization of a later resource fails, already initialized earlier resources are closed. If the body throws and closing also throws, the body exception remains primary and close failures are attached through `getSuppressed()`. If the body succeeds but close fails, the close exception propagates. Multiple close failures retain the first propagating close failure as primary and later ones as suppressed according to reverse close order.

Since Java 9, an existing final or effectively final resource variable can appear directly in the resource specification:

JAVA

```java
var reader = java.nio.file.Files.newBufferedReader(path);
try (reader) {
    System.out.println(reader.readLine());
}
```

The statement assumes ownership and closes it. The variable remains in scope afterward but denotes a closed resource; do not use it.

The conceptual compiler translation uses nested try/finally logic and `addSuppressed`. Writing that translation manually is error-prone, so use try-with-resources whenever ownership is lexical.

Rethrow, wrapping, and interruption

Use `throw ex;` to rethrow the same object and preserve its trace. Wrapping with `new DomainException(message, ex)` creates an abstraction boundary with a cause. `throw new DomainException(ex.getMessage())` discards the cause and should usually be rejected in review.

Do not catch `Throwable` to continue normal service; it includes serious `Error` conditions. Catching `Exception` at a request or job boundary can be appropriate to log, translate, and isolate one unit of work, provided fatal conditions and cancellation semantics remain intact.

`InterruptedException` communicates cooperative cancellation. Code that cannot propagate it should normally restore the status with `Thread.currentThread().interrupt()` and stop its work. Swallowing it can make shutdown and timeouts fail.

TRACE IT | Worked Java example

This example demonstrates primary and suppressed failures with two resources.

JAVA

```java
public class SuppressionDemo {
    static final class Resource implements AutoCloseable {
        private final String name;

        Resource(String name) {
            this.name = name;
            System.out.println("open " + name);
        }

        @Override
        public void close() {
            System.out.println("close " + name);
            throw new IllegalStateException("close failed: " + name);
        }
    }

    public static void main(String[] args) {
        try (Resource first = new Resource("first");
             Resource second = new Resource("second")) {
            throw new IllegalArgumentException("body failed");
        } catch (Exception ex) {
            System.out.println("primary: " + ex.getMessage());
            for (Throwable suppressed : ex.getSuppressed()) {
                System.out.println("suppressed: " + suppressed.getMessage());
            }
        }
    }
}
```

The body failure stays primary, preserving the operation's causal event. Both cleanup failures remain discoverable for diagnosis.

TRACE IT | Execution or memory walkthrough

`first` initializes, then `second`. The body constructs and throws `IllegalArgumentException`. Before catch selection, resources close in reverse order. Closing `second` throws; because a body exception is already pending, this failure is added to the body's suppressed list. Closing `first` also throws and is added next.

The `catch (Exception ex)` matches the primary `IllegalArgumentException`. It prints `body failed`, followed by `close failed: second` and `close failed: first`. The close exceptions are not causes; they are independent secondary failures during cleanup.

Had resource construction for `second` thrown, the body would never begin, `first` would still close, and any close failure would be suppressed under the second-construction failure.

Complexity and performance

Normal try blocks do not inherently change algorithmic complexity. Throwing and capturing a stack trace can be expensive relative to ordinary branches, and exceptions should not represent expected high-volume control flow. Catch matching scales with propagation depth and handlers but is not usually the dominant production concern.

Try-with-resources adds close work that was required anyway. Its safety far outweighs tiny structural overhead. Stack traces and retained causes consume memory, especially if exceptions are accumulated rather than logged and released.

> **HotSpot note:** HotSpot optimizes normal paths through exception regions and may omit or change some diagnostic detail for certain repeated VM-thrown exceptions. Do not depend on such implementation behavior; preserve useful application context explicitly.

WATCH OUT | Edge cases and common mistakes

- Catching a superclass before its subclass is unreachable code.
- Throwing from `finally` masks a pending exception; returning from `finally` masks both exceptions and returns.
- A `throws Exception` declaration exports implementation uncertainty and burdens every caller.
- Logging and rethrowing at every layer produces duplicate noise; log where context and ownership meet.
- Wrapping without a cause destroys the diagnostic chain.
- Catching and ignoring `InterruptedException` breaks cancellation.
- Retrying after an unknown partial side effect can duplicate writes.
- `AutoCloseable.close` is allowed to throw broad `Exception` and is not universally idempotent.
- Resource variables close in reverse declaration order.
- A caller-supplied stream should not be closed by a callee unless ownership transfer is explicit.
- `getCause()` and `getSuppressed()` represent different relationships.
- An exception message can leak tokens, SQL, customer data, or filesystem paths.

Production engineering notes

Define exceptions at architectural boundaries. Translate driver errors into repository failures and domain failures into stable API responses, while retaining the original cause internally. Give clients machine-readable error codes rather than parsing messages.

Retries require classification, idempotency, deadlines, bounded attempts, backoff, and jitter. Never classify only by broad exception type when the provider exposes more specific status. Preserve interruption and cancellation.

Use try-with-resources for files, JDBC statements/results, streams, locks represented by closeable guards, and telemetry scopes. Declare ownership in method names or documentation. At top-level request and worker boundaries, record exception class, safe context, trace or correlation ID, and suppressed failures without logging secrets.

TRY IT | Interview questions and model answers

What is the difference between checked and unchecked exceptions?

Checked exceptions are `Exception` subclasses other than `RuntimeException`; Java requires callers to catch or declare them. Runtime exceptions and errors are unchecked. The hierarchy affects compile-time handling, not an absolute recoverability classification.

What happens if both a try body and close throw?

In try-with-resources, the body exception remains primary. Close exceptions are attached as suppressed throwables in reverse close order. This preserves both the causal operation failure and cleanup evidence.

Why should you avoid return in finally?

It replaces a pending return or exception, silently changing behavior and losing diagnostics. `finally` should perform limited cleanup that does not determine the method outcome.

When should an exception be translated?

At an abstraction boundary where the lower-level type is not part of the caller's contract. The translated exception should add safe, actionable context and retain the original as its cause.

How should InterruptedException be handled?

Prefer propagating it so an owning layer can cancel. If an API cannot declare it, restore interrupt status, stop the current operation, and translate only if the boundary requires it. Do not swallow and continue.

TRY IT | Exercises

1. Predict primary and suppressed exceptions for three resources whose body and close methods fail.
2. Refactor manual file cleanup into try-with-resources and test initialization failure.
3. Design repository exception translation that preserves SQL cause without exposing SQL to clients.
4. Find all ways a return or throw in `finally` can alter a pending outcome.
5. Implement a retry loop that stops on interruption, deadline, permanent failure, and maximum attempts.
6. Document ownership for a method accepting an `InputStream`, then offer borrowing and ownership-transfer variants.

RECAP | Chapter summary

Exceptions represent abrupt completion and propagate to the first compatible handler. Checked status is a compile-time contract choice. `finally` participates in abrupt completion and can dangerously replace outcomes. Try-with-resources initializes left to right, closes right to left, and preserves secondary failures through suppression. Production failure handling retains causes, respects cancellation and ownership, translates at abstraction boundaries, and retries only with bounded, idempotent policy.

TRY IT | Revision checklist

- [ ] I can classify checked, runtime, and error types.
- [ ] I can trace catch selection and propagation.
- [ ] I distinguish causes from suppressed exceptions.
- [ ] I know try-with-resources initialization and reverse close order.
- [ ] I never return from `finally` and avoid throwing from cleanup.
- [ ] I preserve causes when translating failures.
- [ ] I propagate or restore interruption.
- [ ] I make resource ownership, retries, and client error contracts explicit.

SDE-2 CORE CHAPTER

Chapter 6 - Nested Types, Enums, Annotations, and Reflection

Learning objectives

By the end of this chapter, you should be able to:

- distinguish static nested, inner, local, and anonymous classes and their capture rules;
- design enums with behavior, stable external representation, and collection support;
- define annotations with appropriate targets, retention, defaults, and repeatability;
- inspect and invoke types reflectively while respecting access and module boundaries; and
- decide when metadata-driven runtime behavior is worth its safety and complexity costs.

WHY IT WORKS | Why this matters at SDE-2

Framework-heavy Java uses all four topics. Builders and implementation details use nested types, domain state uses enums, dependency injection and serialization use annotations, and frameworks connect metadata to behavior through reflection. An SDE-2 engineer must understand what the framework is doing when scanning fails, an enum evolves, or reflective access is denied.

These features are also design tools. Used carefully, they localize helper types and declarative metadata. Used casually, they create hidden control flow, memory retention, brittle names, and startup surprises.

WHY IT WORKS | First-principles model

A nested type is declared inside another declaration or block. A static nested class has no implicit enclosing object. A non-static member inner class is associated with an enclosing instance. Local and anonymous classes live inside executable code and can capture final or effectively final local values.

An enum is a special class with a fixed declared set of named instances. Each constant is constructed during enum class initialization. An annotation is structured metadata attached to supported program elements or type uses. Its retention policy determines whether it reaches source tools, class files, or runtime reflection.

Reflection treats types and members as data through `Class`, `Method`, `Field`, `Constructor`, and related APIs. It can discover and invoke behavior dynamically, but it shifts checks from compilation to runtime and remains constrained by access control and modules.

> **Specification boundary:** Java defines nested-class semantics, enum identity and initialization, annotation metadata, and reflection APIs. Compiler-generated class names, synthetic capture-field names, framework scan order, and reflective inflation or accessor implementation are not stable language contracts.

Core terminology

- **Static nested class:** nested class with no implicit outer instance.
- **Inner class:** non-static nested class associated with an enclosing instance.
- **Local class:** named class declared inside a block.
- **Anonymous class:** unnamed class expression that extends or implements one type.
- **Capture:** retaining values from an enclosing lexical scope.
- **Enum constant:** canonical instance declared in an enum body.
- **Annotation element:** parameterless metadata member with a restricted return type.
- **Retention:** source-only, class-file, or runtime availability.
- **Type-use annotation:** annotation applicable wherever a type is used.
- **Reflective access:** discovering or operating on program structure at runtime.

Detailed mechanics

Member nested and inner classes

JAVA

```java
class Cache {
    private final String region;

    Cache(String region) { this.region = region; }

    static final class Key {
        private final String value;
        Key(String value) { this.value = value; }
    }

    final class Entry {
        String description() { return region + " entry"; }
    }
}
```

`Cache.Key` can be constructed without a `Cache`: `new Cache.Key("k")`. It cannot directly access an instance field because it has no implicit receiver. `Entry` requires an enclosing instance: `cache.new Entry()`. Its methods can access outer private state.

Use a static nested class unless the semantic relationship truly requires an outer instance. An inner instance retains its enclosing instance, which can unintentionally keep a large object graph alive. Static declarations inside inner classes have become more permissive in modern Java, but an explicit outer relationship is still the defining semantic distinction.

Nested types follow access control and can access private members of their nest under language rules. At the JVM level, modern class files use nestmate metadata to support this without exposing those members publicly.

Local and anonymous classes

A local class is visible only in its declaring block. An anonymous class combines allocation with an unnamed subclass or interface implementation:

JAVA

```java
Runnable task = new Runnable() {
    @Override
    public void run() {
        System.out.println("running");
    }
};
```

Anonymous classes can declare fields and methods but no explicit constructor, and their unique runtime type is awkward to name. Lambdas are usually clearer for functional interfaces, but anonymous classes differ: `this` refers to the anonymous object, they can hold extra state, and they create an actual class instance with class-style semantics.

Local variables captured by local/anonymous classes or lambdas must be final or effectively final. The code captures a value, not a mutable local variable slot. Mutable objects referenced by the captured value can still change.

Enums

Every enum implicitly extends `Enum<E>`, cannot extend another class, and may implement interfaces. Constants are public static final instances. Enum constructors are never public or protected and run during class initialization.

JAVA

```java
enum Severity {
    INFO(1), WARNING(2), CRITICAL(3);

    private final int rank;

    Severity(int rank) { this.rank = rank; }
    int rank() { return rank; }
}
```

Compare enum constants with `==`; identity is the intended semantics and null-safe when the known constant is on the left. `name()` is the declared source name, `ordinal()` is its position, and `toString()` may be

overridden. Never persist ordinal because reordering or inserting constants changes it. Persist an explicit stable code if external compatibility matters.

`Enum.valueOf` matches the exact declared name and throws for unknown text. `values()` returns a new array each call. `EnumSet` and `EnumMap` are specialized, type-safe collections and are preferable to manual bit masks or general-purpose maps for enum domains.

Constants may have constant-specific class bodies that override behavior, but a large behavioral enum can become a closed service locator. Keep behavior cohesive and consider strategies when implementations need independent deployment.

Switches over enums should account for evolution. An exhaustive switch expression without default helps recompilation reveal a new constant; separately compiled old code may still encounter a defensive runtime failure with a newer enum.

Declaring annotations

JAVA

```java
import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
public @interface Operation {
    String value();
    boolean idempotent() default false;
}
```

Annotation elements may return primitives, `String`, `Class`, enum, annotation, or one-dimensional arrays of these. They have no parameters or throws clauses. Values must be compile-time-compatible annotation values; null is not allowed. Defaults belong to the annotation method declaration and are applied when read, so changing a default can change behavior of already compiled annotated code at runtime.

Important meta-annotations include:

- `@Target` for legal declaration or type-use sites;
- `@Retention` with SOURCE, CLASS, or RUNTIME;
- `@Documented` for API documentation inclusion;
- `@Inherited`, which applies only to class-level annotation lookup through superclass inheritance, not interfaces, members, or general meta-annotation composition; and
- `@Repeatable`, whose value names a container annotation.

Runtime processors need RUNTIME retention. Compile-time annotation processors usually work from source/model APIs and do not require runtime retention. TYPE_USE supports nullness and other type-system qualifiers, while TYPE targets a type declaration.

Reflection

Obtain a `Class<?>` from a literal (`Order.class`), object (`value.getClass()`), primitive literal (`int.class`), array class, or `Class.forName`. A class literal normally does not initialize the class; certain reflective uses and `Class.forName(String)` default behavior can initialize it.

`getMethods()` returns public methods including inherited ones. `getDeclaredMethods()` returns methods declared directly regardless of access, excluding inherited methods. Similar distinctions apply to fields and constructors.

JAVA

```
Operation metadata = method.getAnnotation(Operation.class);
Object result = method.invoke(target, argument);
```

Reflective invocation wraps an exception thrown by the target in `InvocationTargetException`; inspect its cause. Argument mismatch and access failures are runtime exceptions or checked reflective failures rather than compile-time errors.

Calling `setAccessible(true)` or `trySetAccessible()` does not grant unlimited authority. In Java 17 and 21, strong module encapsulation can deny deep reflection when the declaring package is not opened to the caller's module. Public access may require package export; deep reflective access usually requires `opens`, command-line configuration, or an API designed for it.

Method handles in `java.lang.invoke` offer a typed, composable alternative and often integrate better with JVM optimization, but lookup access rules still apply. Reflection is appropriate for frameworks and tools, not as a replacement for ordinary polymorphism when types are known.

TRACE IT | Worked Java example

This small registry discovers annotated methods on an explicitly supplied handler. It avoids uncontrolled classpath scanning.

JAVA (continued 1/2)

```
import java.lang.annotation.*;
import java.lang.reflect.*;
import java.util.*;

public class OperationRegistry {
    @Retention(RetentionPolicy.RUNTIME)
    @Target(ElementType.METHOD)
    @interface Operation {
        String value();
    }

    static final class Handler {
        @Operation("health")
        public String health() { return "ok"; }
    }

    static Map<String, Method> discover(Class<?> type) {
        Map<String, Method> result = new HashMap<>();
        for (Method method : type.getDeclaredMethods()) {
```

JAVA (continued 2/2)

```
            Operation operation = method.getAnnotation(Operation.class);
            if (operation == null) continue;
            if (method.getParameterCount() != 0 || method.getReturnType() != String.class) {
                throw new IllegalArgumentException("invalid operation method: " + method);
            }
            if (result.putIfAbsent(operation.value(), method) != null) {
                throw new IllegalArgumentException("duplicate operation: " + operation.value());
            }
        }
        return Map.copyOf(result);
    }

    public static void main(String[] args) throws ReflectiveOperationException {
        Handler handler = new Handler();
        Method method = discover(Handler.class).get("health");
        System.out.println(method.invoke(handler));
    }
}
```

The nested annotation and handler are scoped to the example. Production registries should validate public accessibility, inherited-method policy, duplicate names, return contracts, and target exceptions deliberately.

TRACE IT | Execution or memory walkthrough

Loading `OperationRegistry` makes its nested class metadata available. `Handler.class` supplies a class object without constructing a handler. `getDeclaredMethods` creates reflection descriptors for directly declared methods; their order is unspecified, so the registry must not depend on iteration order.

`getAnnotation` returns a runtime annotation representation because retention is RUNTIME. The registry validates the signature and stores the `Method` under `health`, then creates an unmodifiable map. `method.invoke(handler)` checks receiver and arguments and dispatches to `health`, which returns `"ok"`.

If `health` threw, `invoke` would throw `InvocationTargetException` with the application exception as cause. If the method were inaccessible across a module boundary, discovery might still see a declared descriptor, but invocation could fail unless access was legitimately available.

Complexity and performance

Enum comparison and ordinal-based specialized collection operations are generally O(1). `values()` copies O(n) constants. Reflection discovery is O(m) in inspected members plus annotation and validation work. Cache validated descriptors rather than scanning on each request.

Reflective invocation adds access checks, argument adaptation, boxing, and indirection compared with a direct call. For startup wiring or administrative paths this is usually irrelevant. For a hot serializer loop, cache method handles or generated accessors and benchmark realistic data.

> **HotSpot note:** Reflection and method-handle implementations can optimize repeated access, and the JIT may inline adapted method-handle chains. Exact thresholds and generated accessor strategies are implementation-specific and version-dependent.

WATCH OUT | Edge cases and common mistakes

- A non-static inner instance retains an enclosing instance.
- Captured locals must be effectively final, but referenced mutable state can still race.
- Anonymous-class `this` differs from lambda `this`.
- Persisting enum ordinal breaks when constants are reordered.
- Persisting `toString()` breaks when presentation changes; use an explicit stable code.
- `Enum.valueOf` is case-sensitive and rejects unknown future values.
- Runtime reflection cannot see annotations with SOURCE or CLASS retention through ordinary annotation APIs.
- `@Inherited` does not propagate method annotations or interface annotations.
- Reflection member order is unspecified.
- `getMethod` and `getDeclaredMethod` differ in inheritance and access scope.
- Target exceptions arrive wrapped by `InvocationTargetException`.
- Strong module encapsulation can reject deep reflection despite `setAccessible`.
- Annotation defaults can change runtime interpretation without recompiling clients.

Production engineering notes

Prefer static nested types for builders and implementation helpers. Make inner ownership explicit and avoid registering long-lived callbacks that accidentally retain an outer service. Captured state used concurrently needs the same synchronization as any other shared state.

Give externally serialized enums stable codes and an unknown-value policy. Adding a constant is source-compatible in many locations but can break exhaustive consumers, database constraints, or generated clients. Roll out readers before writers when protocols evolve.

Treat annotations as an API. Specify valid placement, inheritance/merge behavior, duplicates, defaults, and validation timing. Fail fast at startup rather than on the first request. Limit reflection to known packages or explicit registrations, cache results, respect modules, and never invoke untrusted member names without authorization and validation.

TRY IT | Interview questions and model answers

What is the difference between a static nested class and an inner class?

A static nested class has no implicit enclosing instance and cannot directly use outer instance state. A member inner class is associated with a particular outer object and can access it, which also means it retains that object.

Why are captured local variables effectively final?

The nested object may outlive the method activation, so it captures the local's value rather than sharing its stack variable slot. Reassignment would create ambiguous value semantics. The captured reference can still designate mutable data.

Why should enum ordinal not be persisted?

Ordinal is declaration position, not a stable domain code. Inserting or reordering constants changes it. Persist an explicit immutable code and map unknown values intentionally.

What does runtime retention mean?

The annotation is recorded in the class file and exposed through runtime reflection APIs. CLASS retention records it but ordinary runtime reflection does not expose it; SOURCE retention need not enter the class file.

Can reflection access every private field?

No. Access depends on the caller, language access, security context, and module openness. Strong encapsulation in Java 17 and 21 can prevent deep reflection unless the package is opened appropriately.

TRY IT | Exercises

1. Demonstrate an inner callback retaining its outer object, then convert it to a static nested class with explicit state.
2. Define an enum with stable external codes and safe unknown-code parsing.
3. Create a repeatable runtime method annotation and inspect both repeated instances.
4. Extend the registry to unwrap `InvocationTargetException` while preserving its cause.
5. Compare `getMethods` and `getDeclaredMethods` on a subclass with public and private members.
6. Design a startup validation policy for annotated handlers covering duplicates, visibility, signatures, and module access.

RECAP | Chapter summary

Nested types control lexical scope and object relationships; only inner forms carry an implicit enclosing instance. Enums are canonical typed instances and need explicit external codes for evolution. Annotations provide constrained metadata whose target and retention define its reach. Reflection connects metadata to runtime behavior but moves checks later and remains subject to access and modules. Use explicit registration, validation, caching, and narrow authority to keep dynamic systems understandable.

TRY IT | Revision checklist

- [ ] I distinguish static nested, inner, local, and anonymous classes.
- [ ] I understand capture, effectively final locals, and outer-instance retention.
- [ ] I use enum identity safely and never persist ordinal.
- [ ] I choose annotation targets, retention, defaults, and repeatability deliberately.
- [ ] I know the limits of `@Inherited`.
- [ ] I distinguish declared from inherited reflection queries.
- [ ] I unwrap target failures and respect module encapsulation.
- [ ] I validate and cache reflective metadata outside hot request paths.

SDE-2 CORE CHAPTER

Chapter 7 - Generics, Variance, Type Erasure, and Heap Pollution

Learning objectives

By the end of this chapter, you should be able to:

- design generic classes and methods with useful bounds;
- explain invariance and apply upper- and lower-bounded wildcards;
- reason about wildcard capture and type inference;
- describe erasure, bridge methods, reifiability, and generic restrictions; and
- prevent raw-type leaks, unsafe generic arrays, and heap pollution.

WHY IT WORKS | Why this matters at SDE-2

Generics make collection and framework APIs reusable while preserving type safety. Weak generic design forces casts onto callers; overly clever signatures become unreadable. SDE-2 engineers must balance precision and usability, especially in repositories, event buses, serializers, and reusable algorithms.

Interview questions often focus on why `List<Integer>` is not a `List<Number>`, what `? super T` permits, and how erasure causes bridge methods or heap pollution. The production version is harder: warnings appear far from the eventual `ClassCastException`.

WHY IT WORKS | First-principles model

A type parameter is a compile-time variable ranging over reference types. A parameterized type such as `List<String>` states that operations observe a consistent element type. Java implements generics primarily through erasure: most type arguments do not exist as independently testable runtime types.

Generic classes are invariant by default. Even though `Integer` is a subtype of `Number`, `List<Integer>` is not a subtype of `List<Number>`, because the latter would allow insertion of a `Double`. Wildcards express use-site variance: `? extends Number` is an unknown specific subtype useful for producing numbers, and `? super Integer` is an unknown specific supertype useful for consuming integers.

Heap pollution occurs when a variable of a parameterized type refers to an object that does not satisfy that parameterization. It usually begins with an unchecked operation and fails later when a compiler-inserted cast encounters the wrong value.

> **Specification boundary:** Java specifies generic typing, erasure translation, casts, and bridge behavior. It does not preserve arbitrary type arguments for runtime `instanceof`, and reflective generic signatures are metadata rather than a fully reified runtime type system.

Core terminology

- **Type parameter:** declared type variable such as `<T>`.
- **Type argument:** supplied type such as `String` in `List<String>`.
- **Parameterized type:** generic declaration instantiated with arguments.
- **Bound:** restriction such as `<T extends Comparable<? super T>>`.
- **Invariance:** no subtype relation follows between parameterizations merely from their arguments.
- **Wildcard:** unknown type argument written `?` with optional upper or lower bound.
- **Capture conversion:** compiler treats a wildcard as a fresh unknown type.
- **Erasure:** mapping generic types to non-generic runtime representations with casts as needed.
- **Reifiable type:** type whose runtime representation retains enough information for checks.
- **Heap pollution:** runtime value violates a parameterized variable's promised type.

Detailed mechanics

Generic declarations and methods

JAVA

```java
final class Box<T> {
    private final T value;

    Box(T value) { this.value = value; }
    T value() { return value; }
}

static <T> T first(java.util.List<T> values) {
    if (values.isEmpty()) throw new IllegalArgumentException("empty");
    return values.get(0);
}
```

`T` is in scope for instance members of `Box`, but static members cannot use the class's `T` because static state is shared across all parameterizations. A static method may declare its own `<T>` before the return type.

Primitive types cannot be type arguments, so use wrappers or primitive-specialized APIs. `Box<String>` and `Box<Integer>` share the same raw runtime class object under erasure.

Type inference uses argument context, target type, and bounds. Explicit type witnesses such as `Collections.<String>emptyList()` are occasionally useful when inference lacks context, but modern Java resolves most ordinary calls.

Bounds

An upper bound exposes operations supported by the bound:

```java
static <T extends Number> double total(java.util.List<T> values) {
    double sum = 0;
    for (T value : values) sum += value.doubleValue();
    return sum;
}
```

Multiple bounds use one class first, followed by interfaces: `<T extends Base & Comparable<T> & Serializable>`. Erasure of a type variable is its leftmost bound, which makes bound order relevant to generated binary signatures.

Recursive or F-bounds describe operations in terms of the implementing type. A common ordering bound is `<T extends Comparable<? super T>>`, which accepts a type comparable to itself or one of its supertypes. This is more flexible than `Comparable<T>` for inherited comparison definitions.

Invariance and wildcards

Given `List<Integer> integers`, assignment to `List<Number>` is illegal. Both can, however, be viewed through `List<? extends Number>`:

```java
java.util.List<? extends Number> source = java.util.List.of(1, 2, 3);
Number number = source.get(0);
// source.add(4); // rejected; exact captured subtype is unknown
```

You cannot add any non-null value because the actual list might be `List<Double>`. Null is type-compatible but adding it is rarely useful.

For a consumer:

```java
java.util.List<? super Integer> target = new java.util.ArrayList<Number>();
target.add(10);
Object value = target.get(0);
```

The target is known to accept `Integer`, but reading yields only `Object` because its exact element type might be `Object` or `Number`. The mnemonic PECS is useful: producer extends, consumer super. It describes the role of a parameter, not an absolute law. A mutable parameter used for both reading and writing often needs an exact `List<T>`.

An unbounded `List<?>` means a list of some one unknown type, not a list that can contain any types. It is safer than raw `List`: reads yield `Object`, and arbitrary writes are rejected.

API variance examples

JAVA

```java
static <T> void copy(
        java.util.List<? extends T> source,
        java.util.List<? super T> destination) {
    for (T element : source) {
        destination.add(element);
    }
}
```

This accepts `List<Integer>` into `List<Number>` or `List<Object>`. If both parameters were `List<T>`, invariant inference would reject useful combinations.

Return types usually should not expose wildcards when the method can state a concrete parameterization. Returning `List<? extends Animal>` makes callers handle an unknown type; `List<Animal>` or a method type variable often gives a clearer ownership contract.

Wildcard capture

This seemingly simple method fails if written directly because two calls to `list.get` and `list.set` must agree on the captured unknown type. A helper captures it:

JAVA

```java
static void reverseFirstTwo(java.util.List<?> list) {
    reverseFirstTwoCaptured(list);
}

private static <T> void reverseFirstTwoCaptured(java.util.List<T> list) {
    if (list.size() >= 2) {
        T first = list.get(0);
        list.set(0, list.get(1));
        list.set(1, first);
    }
}
```

The public API correctly accepts a list of any single element type. The private method gives that captured type a name so read values can be written back safely.

Erasure and bridge methods

Erasure maps an unbounded type variable to `Object`, a bounded variable to its leftmost bound, and a parameterized type to its raw declaration. The compiler inserts casts where a caller expects a more specific result.

JAVA

```java
interface Provider<T> { T get(); }
final class TextProvider implements Provider<String> {
    public String get() { return "text"; }
}
```

After erasure, `Provider.get` returns `Object`, while the implementation returns `String`. The compiler emits a synthetic bridge method equivalent in effect to `Object get()` delegating to `String get()`, preserving polymorphism. Reflection and stack traces can expose bridge methods; tools should check `Method.isBridge()` and `isSynthetic()` when appropriate.

Two overloads whose signatures erase identically cannot coexist, such as `process(List<String>)` and `process(List<Integer>)`. Neither can code test `value instanceof List<String>` because runtime sees only `List`; `value instanceof List<?>` is legal.

Reifiability and arrays

Primitive types, non-generic classes, raw types, unbounded-wildcard parameterizations, and certain arrays of reifiable types are reifiable. `List<String>` is not. Arrays are reified and covariant, which conflicts with erased invariant generics. Therefore `new T[10]` and `new List<String>[10]` are illegal.

A generic data structure may allocate `Object[]` internally and cast at a controlled boundary, accept an array constructor such as `IntFunction<T[]>`, or prefer collections. Any unchecked cast must be locally justified and representation-safe.

Raw types and heap pollution

Raw types exist for backward compatibility and erase parameter checks:

JAVA

```
java.util.List<String> names = new java.util.ArrayList<>();
java.util.List raw = names;
raw.add(42);                        // unchecked warning
// String first = names.get(0);   // ClassCastException here
```

The list object now violates the promise `List<String>`. The failure appears at the later compiler-inserted cast, not at the unsafe write. Treat unchecked warnings as defects unless a narrow adapter proves safety.

Generic varargs add an array boundary. A `T...` parameter has an array runtime representation that cannot fully represent non-reifiable `T`. `@SafeVarargs` may be applied to eligible methods and constructors only when the body neither corrupts nor unsafely exposes the array.

TRACE IT | Worked Java example

This generic maximum works for types comparable to a supertype and accepts any producing collection.

JAVA

```java
import java.util.Collection;
import java.util.List;
import java.util.NoSuchElementException;

public class GenericMaximum {
    static <T extends Comparable<? super T>> T max(
            Collection<? extends T> values) {
        var iterator = values.iterator();
        if (!iterator.hasNext()) {
            throw new NoSuchElementException("empty input");
        }
        T best = iterator.next();
        while (iterator.hasNext()) {
            T candidate = iterator.next();
            if (candidate.compareTo(best) > 0) {
                best = candidate;
            }
        }
        return best;
    }

    public static void main(String[] args) {
        List<Integer> values = List.of(4, 9, 2);
        Number result = max(values);
        System.out.println(result); // 9
    }
}
```

`Collection<? extends T>` permits a collection of a subtype of the inferred result type. The comparison bound ensures every candidate can compare itself with a compatible supertype.

TRACE IT | Execution or memory walkthrough

For `max(values)`, inference can choose `Integer` for `T`; the target assignment to `Number` also accepts the returned Integer. The iterator has type compatible with the captured producer, and each `next` result can be widened to `T`.

`best` starts as 4. Candidate 9 compares greater and replaces it; candidate 2 does not. The returned reference designates the same immutable Integer object held by the list, not a copy of an object.

At runtime, erasure represents much of this using `Comparable`, `Collection`, and casts inserted at use sites. The static proof prevents unrelated elements from entering through this API. No runtime object representing the type variable `T` is created.

Complexity and performance

Generics do not change the algorithmic complexity of operations. `max` is O(n) time and O(1) additional space. Erasure normally avoids per-parameterization class generation, but wrapper types can introduce boxing and allocation when primitives pass through generic collections.

Bridge calls and casts are small constant costs and are often optimized. Wildcards are compile-time constructs and do not allocate. Prefer primitive-specialized structures only after data volume and profiling justify complexity.

> **HotSpot note:** HotSpot can inline generic erased code, eliminate casts proven redundant, and optimize some boxing. Java generics do not promise specialization for each type argument, and current optimization should not be treated as a no-boxing guarantee.

WATCH OUT | Edge cases and common mistakes

- `List<Dog>` is not a subtype of `List<Animal>`.
- `? extends T` supports safe reads as T but not arbitrary writes.
- `? super T` supports writes of T but reads only as Object.
- `List<?>` is type-safe unknown, while raw `List` disables checks.
- Primitive types cannot be generic arguments.
- Static members cannot use a class's type parameter.
- Overloads cannot differ only by generic arguments with the same erasure.
- `instanceof List<String>` and `new T[]` are illegal because types are not reified.
- An unchecked cast can defer a failure far away from its source.
- `@SuppressWarnings("unchecked")` documents suppression, not proof; keep its scope tiny and comment the invariant.
- `@SafeVarargs` is unsafe if the method writes through an alias or returns its varargs array.
- A wildcard in every return type burdens callers and can indicate an over-general API.
- Reflection's `getGenericType` reads signatures but does not make runtime values generically reified.

Production engineering notes

Expose variance at API boundaries and concrete type parameters within implementations. Accept the least restrictive producer or consumer shape the method truly supports. Keep public signatures readable; a named domain interface is sometimes better than a page of bounds.

Eliminate raw types in modern code. At legacy or reflection boundaries, validate runtime values once, copy into a correctly typed structure, and isolate a justified unchecked cast. Compile with unchecked warnings enabled and fail builds on unexplained new warnings.

Type tokens such as `Class<T>` reify only a raw class and cannot represent `List<String>`. Frameworks use richer type descriptors or anonymous-superclass capture for nested generic types. Understand whether their metadata survives proxies, serialization, and ahead-of-time compilation.

TRY IT | Interview questions and model answers

Why is List not a subtype of List?

If it were, a method receiving `List<Number>` could add a Double to an actual integer list. Invariance prevents that. Use `List<? extends Number>` for a producer view or `List<? super Integer>` for a consumer view.

What does PECS mean?

Producer extends, consumer super. An input that only produces T can use `? extends T`; one that only consumes T can use `? super T`. A structure both read and written as the same exact type often uses `T` directly.

What is type erasure?

The compiler checks generic types, then maps most parameterized uses to raw runtime representations, inserting casts and sometimes bridge methods. This supports binary compatibility but prevents runtime tests of arbitrary type arguments.

What is heap pollution?

A parameterized variable refers to data that violates its claimed type, typically because of a raw type, unchecked cast, or unsafe generic varargs operation. A later generated cast then throws, far from the unsafe operation.

Why does the compiler generate bridge methods?

Erasure can make an overriding method's erased descriptor differ from the generic supertype descriptor. A synthetic bridge with the erased signature delegates to the specific implementation, preserving runtime polymorphism.

TRY IT | Exercises

1. Implement `copy` first with invariant lists, observe rejected calls, then add correct producer/consumer wildcards.
2. Write and invoke a helper that captures `List<?>` to swap two positions.
3. Explain the bound `<T extends Comparable<? super T>>` using a superclass that implements Comparable.
4. Create heap pollution through a raw list and locate the later compiler-inserted cast.
5. Inspect `TextProvider` reflection output and identify its bridge method.
6. Design a type-safe heterogeneous registry using `Class<T>` keys and isolate any checked cast.

RECAP | Chapter summary

Generics provide compile-time consistency over reference types. Parameterizations are invariant, while wildcards express producer and consumer variance. Bounds expose operations and capture helpers name unknown types. Erasure preserves a shared runtime representation, requiring casts and bridge methods and preventing tests of most type arguments. Raw types, unchecked casts, and unsafe generic varargs can pollute the heap; confine unavoidable unchecked boundaries and prove their invariants.

TRY IT | Revision checklist

- [ ] I can declare generic classes, methods, and multiple bounds.
- [ ] I can explain invariance with the unsafe-insertion argument.
- [ ] I apply extends for producers and super for consumers.
- [ ] I understand unbounded wildcard versus raw type.
- [ ] I can use a helper to capture a wildcard.
- [ ] I know erasure, reifiability, and bridge-method consequences.
- [ ] I can identify every common source of heap pollution.
- [ ] I isolate and justify unavoidable unchecked operations.

SDE-2 CORE CHAPTER

Chapter 8 - Lambdas, Method References, and Functional Interfaces

Learning objectives

By the end of this chapter, you should be able to:

- define and recognize functional interfaces, including generic primitive variants;
- explain lambda target typing, capture, scope, and exception compatibility;
- distinguish the four principal method-reference forms;
- compose behavior while controlling side effects and concurrency assumptions; and
- design functional APIs that remain readable, testable, and allocation-aware.

WHY IT WORKS | Why this matters at SDE-2

Modern Java APIs use functions for streams, asynchronous stages, retries, transaction callbacks, and configuration. A lambda can make variation concise, but it can also hide blocking work, capture mutable state, or make exception handling opaque.

SDE-2 interviews test syntax less often than semantics: whether a lambda is an object, which overload it targets, how `this` behaves, and why a captured counter will not compile. Production reviews add ownership, threading, observability, and idempotency.

WHY IT WORKS | First-principles model

A lambda expression is source syntax for supplying an implementation of a functional interface's single abstract method. It has no standalone type. Its target type comes from assignment, invocation, cast, or return context, and that context determines parameter and return compatibility.

A method reference is another compact implementation form that delegates the functional method to an existing static method, bound receiver, unbound receiver, or constructor. Neither syntax promises a particular generated class, allocation, or object identity.

Captured local values must be final or effectively final. The lambda captures their values, not mutable stack slots. A captured reference may still reach mutable state, so type legality does not imply thread safety.

> **Specification boundary:** Java specifies lambda behavior through target functional interfaces, including capture and `this` semantics. The runtime representation, caching, generated class shape, and use of `invokedynamic` linkage are implementation details and must not be observed through identity assumptions.

Core terminology

- **Functional interface:** interface with one abstract method after inheritance and override-equivalence rules.
- **SAM:** single abstract method and its function type.
- **Target typing:** surrounding context determines a lambda or method-reference type.
- **Capture:** lambda retains values from its lexical scope.
- **Effectively final:** local assigned once even without `final` modifier.
- **Bound method reference:** receiver object is captured, such as `printer::print`.
- **Unbound method reference:** receiver becomes the first function parameter, such as `String::length`.
- **Constructor reference:** creates through `Type::new` or an array constructor.
- **Higher-order function:** accepts or returns behavior.
- **Non-interference:** operation does not modify the source or shared state in a way that invalidates processing.

Detailed mechanics

Functional interface rules

`@FunctionalInterface` makes the compiler validate intent. The annotation is optional but useful. Default, static, and private methods do not count as abstract methods. Public methods corresponding to `Object` methods do not prevent functional status in the relevant interface rules.

JAVA

```java
@FunctionalInterface
interface CheckedSupplier<T> {
    T get() throws Exception;

    default T getOrElse(T fallback) {
        try {
            return get();
        } catch (Exception ex) {
            return fallback;
        }
    }
}
```

Generic interface methods can prevent lambda targeting in cases where no single concrete function type can be inferred. Keep custom interfaces focused and reuse `java.util.function` when its naming and exception contract fit.

Common interfaces include:

- `Predicate<T>`: T to boolean;
- `Function<T,R>`: T to R;
- `UnaryOperator<T>`: T to T;
- `BinaryOperator<T>`: two Ts to T;
- `Consumer<T>`: T to void;
- `Supplier<T>`: no arguments to T; and
- primitive specializations such as `IntPredicate`, `ToLongFunction<T>`, and `LongSupplier` to reduce boxing.

Lambda syntax and target typing

JAVA

```
java.util.function.Predicate<String> nonBlank = s -> !s.isBlank();
java.util.function.BinaryOperator<Integer> add = (left, right) -> left + right;
java.util.function.Supplier<String> empty = () -> "";
```

One inferred parameter can omit parentheses. Multiple or zero parameters require them. A block body uses statements and must return on every normal path when the functional result is non-void. Parameter types must be all inferred, all `var`, or all explicit; annotations can be placed on `var` parameters.

The same lambda text can target different interfaces. Overloads taking unrelated functional interfaces with identical-looking shapes can therefore be ambiguous:

JAVA

```
static void use(java.util.function.Function<String, Integer> f) {}
static void use(java.util.function.ToIntFunction<String> f) {}
// use(s -> s.length()); // ambiguous
use((java.util.function.ToIntFunction<String>) String::length);
```

Avoid public overload sets distinguished only by similar function types. Named methods or differently named operations are easier for callers.

Scope, capture, and this

JAVA

```
int limit = 10;
java.util.function.IntPredicate small = value -> value < limit;
// limit = 20; // would make limit not effectively final
```

The value 10 is captured. Instance lambdas can access fields, whose values may change because the lambda captures `this`, not a snapshot of each field. Local variables cannot be redeclared with the same name inside a lambda parameter or body when lexical scope forbids shadowing.

Lambda `this` and `super` are lexically scoped: they refer to the enclosing instance. In an anonymous class, `this` refers to the anonymous object. This distinction matters for listeners and recursion.

Capture can extend object lifetimes. Storing a lambda that references a large service or request object retains that graph. A stateless lambda may be reused by an implementation, but code must not depend on reuse or synchronize on a lambda object.

Method references

Four common forms are:

JAVA

```
java.util.function.IntBinaryOperator max = Math::max;       // static
var prefix = "id:";
java.util.function.Function<String, String> add = prefix::concat; // bound
java.util.function.ToIntFunction<String> length = String::length;  // unbound
java.util.function.Supplier<java.util.ArrayList<String>> list =
        java.util.ArrayList::new;                            // constructor
```

For `String::length`, the function argument is the receiver. An unbound reference can also have additional parameters, as `String::indexOf` matching `(String receiver, String search) -> int`, subject to overload resolution.

A bound receiver expression is evaluated when the method reference is created, not each time it is invoked. If that expression is null, creation can throw `NullPointerException`. A lambda `x -> receiver.method(x)` evaluates its captured receiver according to its own expression semantics and may fail later, so the forms are not always timing-equivalent.

Array constructor references such as `String[]::new` match `IntFunction<String[]>` and preserve a reified component type.

Composition

Standard interfaces provide combinators. Predicates support `and`, `or`, and `negate`; functions support `compose` and `andThen`; consumers support sequential `andThen`.

JAVA

```
java.util.function.Function<String, String> strip = String::strip;
java.util.function.Function<String, String> lowercase =
        text -> text.toLowerCase(java.util.Locale.ROOT);
var normalize = strip.andThen(lowercase);
```

`strip.andThen(lowercase)` applies strip first. `strip.compose(lowercase)` would apply lowercase first. Composition preserves thrown runtime exceptions and does not introduce rollback; if the first consumer has a side effect and the second fails, the first effect remains.

Standard function interfaces do not declare checked exceptions. Options are to handle within the lambda, adapt from a custom checked interface at a boundary, or redesign the surrounding API. A generic "sneaky throw" erodes declared contracts and should not be a default strategy.

Behavior and concurrency

A callback's contract must say whether it may run zero, one, or many times; synchronously or asynchronously; sequentially or concurrently; and on which thread or executor. Lambdas do not make captured mutable objects safe. Side effects in parallel streams or retry callbacks can race or repeat.

Do not assume function instances have meaningful `equals`, `hashCode`, serialization, or stable class names. If behavior needs persistent identity, represent it as explicit data or a named strategy type.

TRACE IT | Worked Java example

This validator composes named predicates and returns useful failures rather than a bare boolean.

JAVA

```java
import java.util.List;
import java.util.function.Predicate;

public class ValidationDemo {
    record Rule<T>(String message, Predicate<? super T> test) {}

    static <T> List<String> validate(T value, List<Rule<T>> rules) {
        return rules.stream()
                .filter(rule -> !rule.test().test(value))
                .map(Rule::message)
                .toList();
    }

    public static void main(String[] args) {
        List<Rule<String>> rules = List.of(
                new Rule<>("must not be blank", text -> !text.isBlank()),
                new Rule<>("must be at most 12 characters", text -> text.length() <= 12));

        System.out.println(validate("a very long value", rules));
    }
}
```

`Predicate<? super T>` lets a rule consume T using a predicate defined for T or a supertype. Each rule also carries durable descriptive data rather than requiring introspection of the lambda.

TRACE IT | Execution or memory walkthrough

The two lambda expressions are target-typed as `Predicate<? super String>` through record construction and list inference. The list stores references to two immutable rule records, each of which holds a message and predicate reference.

`validate` creates a lazy stream pipeline. At terminal `toList`, each rule enters the filter. The blank rule passes its test, so negation makes the filter false and its message is excluded. The length rule returns false; negation includes it. `Rule::message` is an unbound instance reference whose input is a Rule receiver. `toList` returns an unmodifiable result list containing the one failure message.

The implementation may allocate or reuse lambda objects, but no correctness depends on their identity. The captured lambdas here are stateless.

Complexity and performance

Creating or invoking functional behavior is conceptually constant overhead; body cost dominates. Validation is O(r) predicate invocations and O(f) result space for r rules and f failures. Predicate order can short-circuit when explicitly composed, while this implementation intentionally evaluates every rule to collect all messages.

Generic standard interfaces can box primitive arguments and results. Primitive specializations reduce that cost on numeric hot paths. Deep chains may affect stack traces and debugging clarity even if the JIT inlines them.

> **HotSpot note:** Java compilers commonly emit `invokedynamic` lambda factories, and HotSpot can reuse stateless instances, inline bodies, and remove allocations. No identity, singleton, or allocation guarantee follows from those optimizations.

WATCH OUT | Edge cases and common mistakes

- A lambda has no standalone type; it needs a target functional interface.
- Similar functional-interface overloads can be ambiguous.
- Captured locals must be effectively final; captured objects may still mutate.
- Lambda `this` is the enclosing `this`, unlike an anonymous class.
- A bound method-reference receiver is evaluated immediately.
- Method references can become unclear when overloads or parameter reordering are involved; use a lambda for clarity.
- Standard function interfaces do not declare checked exceptions.
- Side-effecting callbacks may be invoked multiple times by retries or stream behavior.
- Parallel callbacks can race on captured collections or counters.
- Function-object identity, class names, and serialization are not portable contracts.
- `Consumer.andThen` does not run the second action if the first throws and does not undo completed effects.
- Using `Function<T, Boolean>` instead of `Predicate<T>` introduces boxing and weakens intent.

Production engineering notes

Name nontrivial behavior. A local variable such as `isEligible` or a small strategy class gives stack traces and reviews more meaning than a large inline lambda. Put blocking, transactional, and retry semantics in callback documentation.

Pass immutable captured data where possible. When callbacks run concurrently, use thread-safe collaborators or confine state per invocation. Do not capture request-scoped objects in application-lifetime registries.

Functional interfaces are public APIs. Consider checked failures, variance, invocation count, ordering, nullability, thread, cancellation, and reentrancy. If callers need to log, persist, or configure a policy, pair executable behavior with explicit metadata or use a named domain type.

TRY IT | Interview questions and model answers

Is a lambda an anonymous inner class?

No. Both can implement a functional role, but lambda `this` is lexical, capture and typing rules differ, and the runtime representation is unspecified. An anonymous class introduces a distinct class body and its own `this`.

What is target typing?

The assignment, parameter, cast, or return context supplies the functional interface whose single abstract method determines lambda parameter and result types. Without a target, a lambda expression cannot stand alone.

Why must captured locals be effectively final?

The lambda captures their values and may outlive the method frame. Preventing reassignment preserves clear value capture. It does not freeze objects reached through captured references.

How does `String::length` work as a function?

It is an unbound instance method reference. The function's first and only String argument becomes the receiver, and the result is that receiver's length.

Are lambdas thread-safe?

Not inherently. A stateless lambda can be safely invoked concurrently if its called operations are safe. A lambda capturing mutable state has the same synchronization and race concerns as any other code.

TRY IT | Exercises

1. Write examples of static, bound, unbound, constructor, and array-constructor references.
2. Demonstrate the different meaning of `this` in a lambda and anonymous class.
3. Refactor ambiguous functional overloads into unambiguous method names.
4. Adapt a checked `IOException`-throwing supplier to a domain result without losing cause.
5. Find a lambda capture that retains a request object in a singleton registry and redesign it.
6. Add fail-fast and collect-all modes to `ValidationDemo`, documenting invocation counts.

RECAP | Chapter summary

Lambdas and method references implement target functional interfaces; they are not standalone dynamically typed functions. Capture copies effectively final local values, while object mutability and thread safety remain ordinary concerns. Method references cover static, bound, unbound, and constructor forms, with bound receivers evaluated at creation. Functional API contracts must define exceptions, invocation count, ordering, threading, side effects, and ownership, not only parameter types.

TRY IT | Revision checklist

- [ ] I can determine whether an interface is functional.
- [ ] I understand lambda target typing and overload ambiguity.
- [ ] I can explain local capture and lexical `this`.
- [ ] I know all principal method-reference forms.
- [ ] I use standard and primitive functional interfaces appropriately.
- [ ] I handle checked failures at an explicit boundary.
- [ ] I do not depend on lambda identity, class shape, or serialization.
- [ ] I document callback side effects, repetition, threading, and cancellation.

SDE-2 CORE CHAPTER

Chapter 9 - Java 17 and Java 21 Language and Platform Features

Learning objectives

By the end of this chapter, you should be able to:

- identify the high-impact permanent features introduced in Java 17 and Java 21;
- distinguish permanent, preview, and incubator status as of each release;
- compile and run preview or incubator code with appropriate controls;
- evaluate virtual threads, sequenced collections, pattern matching, and platform changes; and
- plan a Java 17 to Java 21 migration without confusing language compatibility with operational readiness.

WHY IT WORKS | Why this matters at SDE-2

Java 17 and Java 21 are common long-term-support deployment baselines, although support duration is a vendor distribution policy rather than a Java language property. SDE-2 engineers are expected to know which newer constructs improve a service and which are experimental in a given release.

A migration is not complete because source compiles. Strong encapsulation can break reflection, virtual threads can change workload shape, and preview class files require runtime flags.

WHY IT WORKS | First-principles model

Java evolves through language, library, JVM, tooling, security, and platform changes. A feature has a status within a specific release:

- **Permanent:** standard in that release and does not require preview flags.
- **Preview:** fully specified for feedback but not permanent; source and class files require opt-in and may change or disappear.
- **Incubator:** non-final API delivered in a `jdk.incubator.*` module for experimentation; compatibility is not promised.

Preview in one release does not imply the same syntax or API in another. Code must be judged against its exact source release. A feature finalized in Java 21 is ordinary Java 21 even if its ancestors were preview in earlier releases.

> **Specification boundary:** "LTS" is not a Java Language Specification status. Vendors decide support offerings and timelines. Permanent, preview, and incubator status is release-specific; never describe a preview API as standardized merely because a production JDK contains it.

Core terminology

- **Feature release:** numbered Java platform release with its own source/class-file level.
- **LTS:** vendor commitment to extended maintenance for selected releases.
- **Preview feature:** opt-in language or VM feature intended for real-world feedback.
- **Incubator module:** non-final API module shipped for experimentation.
- **Strong encapsulation:** enforcement of module boundaries around JDK internals.
- **Virtual thread:** lightweight Java thread scheduled by the runtime rather than permanently bound one-to-one to an OS thread.
- **Sequenced collection:** collection with defined encounter order and first/last/reversed operations.
- **Pattern matching:** combined testing and extraction based on value shape or type.
- **Generational ZGC:** Z Garbage Collector mode exploiting object-age behavior.
- **Migration baseline:** source, target bytecode, runtime, dependencies, tools, and operations agreed for deployment.

Detailed mechanics

Java 17 permanent features

Java 17 made sealed classes and interfaces permanent. They use `sealed`, `permits`, `final`, and `non-sealed` to define controlled direct inheritance. Pattern matching for `instanceof`, records, text blocks, and switch expressions were already permanent before 17 and are part of a practical Java 17 language baseline.

Java 17 restored always-strict floating-point evaluation. The older `strictfp` distinction became unnecessary for ordinary source semantics: floating-point expressions use consistently strict behavior. Existing `strictfp` code remains source-compatible but the modifier is no longer needed for this purpose.

Java 17 strongly encapsulated JDK internals, with limited critical exceptions. Applications depending on `sun.*` members or illegal deep reflection may fail and should migrate to supported APIs. `--add-opens` can be a temporary operational bridge, not a durable application API.

Java 17 preview and incubator status

As of Java 17:

- Pattern matching for `switch` was **preview**. It required preview flags, and its guarded-pattern syntax and null semantics evolved before finalization.
- The Foreign Function and Memory API was **incubator**, in `jdk.incubator.foreign`.
- The Vector API was in its **second incubator** round, in `jdk.incubator.vector`.

Do not write Java 21 `when` guards and call them Java 17 syntax. Do not assume Java 17 incubator foreign-memory source migrates unchanged to the later `java.lang.foreign` API.

Preview compilation and execution typically use matching JDKs:

BASH

```
javac --enable-preview --release 17 Example.java
java --enable-preview Example
```

Incubator modules require explicit module resolution, for example:

BASH

```
javac --add-modules jdk.incubator.vector VectorExample.java
java --add-modules jdk.incubator.vector VectorExample
```

Build tools, tests, IDEs, packaging, and runtime launchers all need consistent configuration. Preview class files are marked so a runtime does not silently run them without preview opt-in.

Java 21 permanent language and collection features

Java 21 finalized pattern matching for switch. Type patterns, explicit null cases, exhaustive sealed-hierarchy coverage, and `when` guards allow declarative variant handling. It also finalized record patterns, including nested deconstruction. Neither requires `--enable-preview` on Java 21.

JAVA

```
sealed interface Event permits Login, Logout {}
record Login(String user) implements Event {}
record Logout(String user) implements Event {}

static String user(Event event) {
    return switch (event) {
        case Login(String name) -> name;
        case Logout(String name) -> name;
    };
}
```

Java 21 introduced sequenced collection interfaces: `SequencedCollection`, `SequencedSet`, and `SequencedMap`. They provide uniform first, last, and reversed operations for ordered collections. Existing types such as lists, deques, linked hash sets, sorted sets, linked hash maps, and sorted maps participate according to their contracts.

```java
java.util.List<String> queue = new java.util.ArrayList<>();
queue.addLast("second");
queue.addFirst("first");
System.out.println(queue.getFirst());       // first
System.out.println(queue.reversed());       // [second, first]
```

`reversed()` is generally a reverse-ordered view, not an independent deep copy. Mutability and write-through behavior depend on the backing collection's contract.

Virtual threads in Java 21

Virtual threads became permanent in Java 21 after previews in Java 19 and 20. They implement the `Thread` API and are well suited to high-concurrency workloads that spend substantial time blocking on I/O. They preserve a straightforward thread-per-task programming model.

Virtual threads are not automatically faster for CPU-bound work and do not remove downstream capacity limits. Database pools, HTTP peers, file descriptors, memory, and rate limits still need bounds. Do not pool virtual threads merely to reduce thread count; use one per task, then bound the scarce resource itself.

As of Java 21, blocking while holding a monitor in certain operations or executing native/foreign calls can pin a virtual thread to its carrier, reducing scalability. Pinning is a performance condition, not a correctness failure. Measure with Java Flight Recorder and relevant diagnostics, keep synchronized regions short, and avoid holding locks across blocking I/O.

Thread-local values work, but creating millions of virtual threads can make large or mutable thread-local state expensive. Prefer explicit context propagation where practical; scoped values were preview in Java 21 as a structured alternative.

Java 21 JVM and operational changes

Generational ZGC became available in Java 21. It separates young and old generations to exploit the observation that most objects die young. In Java 21 it is selected with ZGC plus the generational option; collector defaults and flag evolution are release-specific, so validate exact deployment commands against the target JDK.

Virtual-thread observability requires tools that understand large thread populations. Do not assume one platform thread per request in dashboards or capacity formulas.

Java 21 preview and incubator status

As of Java 21, the following were **preview**, not permanent:

- String Templates, first preview;
- Unnamed Patterns and Variables, first preview;
- Unnamed Classes and Instance Main Methods, first preview;
- Scoped Values, preview;
- Structured Concurrency, preview; and
- Foreign Function and Memory API, third preview.

The Vector API was in its **sixth incubator** round, not preview or permanent.

String templates combined literal text, expressions, and a processor; preview syntax such as `STR."Hello \{name}"` did not make SQL safe automatically. Unnamed patterns use `_` for intentionally unused bindings. Scoped values provide bounded context, while structured concurrency groups related subtasks and their cancellation. FFM moved to the `java.lang.foreign` API family in Java 21 and was not source-compatible with Java 17's incubator API. All remain subject to their status above.

Migration and compilation boundaries

`--release 17` compiles against the documented Java 17 API and emits compatible class files even when the compiler itself is newer. Merely using `-source 17 -target 17` can accidentally compile against newer boot APIs in some workflows; prefer `--release` when targeting an older platform.

Upgrade in layers:

1. inventory JDK distributions, CPU/OS support, containers, agents, flags, build tools, and libraries;
2. run the existing Java 17 source and bytecode on Java 21 in staging;
3. remove illegal internal API/reflection dependencies and obsolete JVM flags;
4. update CI, static analysis, test frameworks, profilers, and runtime images;
5. establish behavioral and performance baselines; then
6. adopt Java 21 source features deliberately.

Runtime and source-language upgrades need not occur in one deployment.

TRACE IT | Worked Java example

This Java 21 program combines permanent record patterns, pattern switch, and virtual threads. It uses no preview feature.

JAVA

```java
import java.util.List;
import java.util.concurrent.Executors;

public class Java21Demo {
    sealed interface Request permits Read, Write {}
    record Read(String key) implements Request {}
    record Write(String key, String value) implements Request {}

    static String execute(Request request) {
        return switch (request) {
            case Read(String key) -> "read " + key;
            case Write(String key, String value) when value.isBlank() ->
                    "reject blank write to " + key;
            case Write(String key, String value) -> "write " + key + "=" + value;
        };
    }

    public static void main(String[] args) throws Exception {
        List<Request> requests = List.of(new Read("a"), new Write("b", "2"));
        try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
            var futures = requests.stream()
                    .map(request -> executor.submit(() -> execute(request)))
                    .toList();
            for (var future : futures) {
                System.out.println(future.get());
            }
        }
    }
}
```

Compile it with `javac --release 21 Java21Demo.java`; no `--enable-preview` is needed.

TRACE IT | Execution or memory walkthrough

The two records are final permitted implementations. The request list preserves encounter order. Each submitted callable is assigned its own virtual thread by the executor. The runtime schedules virtual threads onto carrier platform threads and can unmount them around supported blocking operations.

The Read task matches and deconstructs its record arm. The Write task first tests the guarded blank arm; the guard is false, so it matches the unguarded Write arm. The main thread obtains futures in list order, so printing is deterministic here even if task completion order differs.

Closing the executor waits for submitted tasks under its `ExecutorService` close contract. A production workflow still needs deadlines, cancellation, partial-failure policy, and downstream concurrency bounds.

Complexity and performance

New syntax does not change underlying algorithmic complexity. Pattern matching replaces explicit tests and casts with compiler-checked forms. Sequenced view operations are typically O(1), but exact operation complexity follows each collection implementation.

Virtual threads reduce the resource cost of large numbers of blocking threads; they do not reduce CPU work or remote latency. Total memory still scales with live task state. Generational ZGC trades CPU, throughput, footprint, and pause characteristics differently from other collectors and must be load-tested with the service's allocation profile.

> **HotSpot note:** Carrier scheduling, virtual-thread continuation representation, JIT decisions, ZGC implementation, and diagnostic event details are HotSpot behavior. Code should rely on Thread and API contracts, while capacity plans should measure the exact JDK build and flags deployed.

WATCH OUT | Edge cases and common mistakes

- LTS is a vendor support designation, not a different language edition.
- Preview source and runtime must use matching release configuration.
- Java 17 pattern switch is preview; Java 21 pattern switch is permanent and uses final `when` guard syntax.
- Record patterns are permanent in 21 and unavailable in 17.
- String templates, scoped values, structured concurrency, unnamed features, and FFM are preview in 21.
- The Vector API is incubator in both releases discussed, at different iterations.
- Virtual threads do not justify unbounded database or network load.
- Long blocking operations while holding monitors can pin carriers in Java 21.
- `reversed()` commonly returns a view, so backing mutations can be visible.
- Strong encapsulation means a successful `--add-opens` workaround is migration debt, not a public contract.
- New JDK execution does not guarantee old agents, flags, or profilers remain compatible.

Production engineering notes

Define one approved JDK distribution and patch policy per environment. Record runtime version, flags, container limits, collector, and agents in deploy metadata. Test upgrades with production-like traffic and compare latency percentiles, CPU, allocation, GC, thread pinning, connection pools, and error rates.

Use virtual threads where blocking code dominates and simpler thread-per-request structure helps. Keep concurrency controls at scarce resources, propagate cancellation, and test every dependency for blocking and thread-local assumptions. Avoid a wholesale asynchronous-to-virtual rewrite without comparative evidence.

Keep preview features behind experiments or internal modules unless the organization accepts upgrade churn and flags across the full toolchain. Never publish a reusable Java 21 library that silently requires preview class files. For permanent features, adopt style guidance so patterns and record deconstruction improve clarity rather than compressing complex business rules.

TRY IT | Interview questions and model answers

Which major language features are permanent in Java 17?

Sealed classes became permanent in 17. Records, text blocks, switch expressions, and pattern matching for `instanceof` had finalized earlier and are available in the Java 17 baseline. Pattern switch itself is only preview in 17.

What became permanent in Java 21?

Pattern matching for switch, record patterns, virtual threads, and sequenced collections are permanent Java 21 features or APIs. They require no preview flag.

Are virtual threads faster than platform threads?

Not inherently. They improve scalability and simplify code for many concurrent blocking tasks by reducing per-thread resource cost. CPU-bound throughput remains bounded by cores, and downstream capacity still needs limits.

What is the difference between preview and incubator?

Preview features are platform language, VM, or API features that require explicit opt-in and may evolve. Incubator APIs live in `jdk.incubator` modules, require module resolution, and are also non-final with intentionally weak compatibility promises.

How would you migrate from 17 to 21 safely?

First run existing source on a Java 21 runtime after updating dependencies, tools, agents, flags, and encapsulation workarounds. Establish functional and performance baselines. Then adopt source features and virtual threads incrementally, with concurrency limits, diagnostics, and rollback.

TRY IT | Exercises

1. Classify every feature in this chapter as permanent, preview, or incubator in Java 17 and Java 21.
2. Compile the Java 21 demo without preview flags and verify its class-file target through `javap -verbose`.
3. Convert an executor-based blocking service to virtual threads while retaining a semaphore around a 20-connection database pool.
4. Replace first/last list indexing utilities with sequenced operations and test empty behavior and reversed-view mutation.
5. Inventory a Java 17 service for `--add-opens`, internal JDK APIs, agents, removed flags, and old build plugins.
6. Design a benchmark and load-test plan comparing platform-thread pools, virtual threads, and existing async code for one real I/O workload.

RECAP | Chapter summary

Java 17 provides a mature baseline with permanent sealed types and strong encapsulation, while its pattern switch is preview and FFM and Vector APIs are incubating. Java 21 permanently adds pattern switch, record patterns, sequenced collections, and virtual threads, alongside operational advances such as generational ZGC. Several Java 21 features remain preview, and Vector remains incubator. Safe adoption separates runtime migration from source migration and validates dependencies, flags, concurrency, observability, and workload performance.

TRY IT | Revision checklist

- [ ] I distinguish vendor LTS policy from Java feature status.
- [ ] I can name permanent Java 17 and Java 21 language features.
- [ ] I accurately label preview and incubator features in each release.
- [ ] I know the compile and runtime controls for preview and incubator code.
- [ ] I understand virtual-thread strengths, limits, pinning, and resource bounds.
- [ ] I understand sequenced collection views and Java 21 pattern syntax.
- [ ] I can explain strong encapsulation and migration debt from add-opens.
- [ ] I can plan a measured Java 17 to 21 rollout before adopting new syntax.

SDE-2 CORE CHAPTER

Chapter 10 - Java 17 and Java 21 Feature Matrix

This matrix emphasizes features that influence SDE-2 interviews and backend engineering. It uses Java 21 as the executable baseline and distinguishes final features from preview or incubating work. A feature can be introduced across several releases before becoming final; the final release is the portable baseline used here.

LTS orientation

Release	Status in this book	Selected significance
Java 8	historical baseline	lambdas, streams, default interface methods, `java.time`, `CompletableFuture`
Java 11	migration baseline	standard HTTP client, single-file launch, module-era runtime, removed bundled technologies
Java 17	supported modern baseline	records, sealed classes, pattern matching for `instanceof`, strong encapsulation context
Java 21	main code baseline	virtual threads, record patterns, pattern switch, sequenced collections, generational ZGC option
Java 25	later ecosystem context	LTS after the book's Java 21 code baseline; do not assume its APIs in examples

Long-term-support labels and commercial support schedules are vendor policy, not Java language semantics. Verify the distribution and support contract used by an employer.

Language feature timeline

Feature	Final release	Java 17	Java 21	Interview implication
local variable type inference (`var`)	10	yes	yes	local static type remains fixed; not dynamic typing
switch expressions	14	yes	yes	value-producing, exhaustive form with arrow labels
text blocks	15	yes	yes	multi-line literals with incidental indentation rules
records	16	yes	yes	nominal data carriers with generated members and shallow immutability
pattern matching for `instanceof`	16	yes	yes	flow-scoped pattern variable
sealed classes/interfaces	17	yes	yes	closed direct-subtype set; supports exhaustive reasoning
record patterns	21	no	yes	nested deconstruction patterns
pattern matching for switch	21	preview	final	type/record patterns, null handling, dominance and exhaustiveness rules

var

JAVA

```java
var names = new java.util.ArrayList<String>();
```

`var` is permitted for qualifying local variables, loop variables, and lambda parameters in specified forms. It cannot declare fields, method parameters, or return types. The initializer determines a compile-time type; `names` is not dynamically typed. Use it when the initializer makes the type clear, not to hide a meaningful abstraction.

Switch expressions

JAVA

```java
int priority = switch (severity) {
    case LOW -> 1;
    case MEDIUM -> 2;
    case HIGH -> 3;
};
```

The compiler checks that a switch expression produces a value for every possible input covered by the type rules. In a block arm, use `yield` to produce the value. The older colon statement form remains and can fall through.

Text blocks

JAVA

```java
String json = """
        {
          "enabled": true
        }
        """;
```

Text blocks reduce escaping but still produce an ordinary `String`. Their indentation and trailing-newline behavior are defined transformations; use `stripIndent` or explicit escapes only when the contract requires it.

Records

JAVA

```java
record Point(int x, int y) {}
```

A record is implicitly final and extends `java.lang.Record`. It receives private final component fields, accessors named after components, a canonical constructor, and value-oriented `equals`, `hashCode`, and `toString`. It may implement interfaces and define members, but cannot declare extra instance fields. Component objects can still be mutable.

Sealed hierarchies

JAVA

```java
sealed interface Command permits Create, Delete {}
record Create(String id) implements Command {}
record Delete(String id) implements Command {}
```

The permits relationship is enforced at compile time and in class metadata. It is useful when the domain genuinely owns a closed alternative set. `non-sealed` deliberately reopens a branch.

Record patterns and pattern switch

JAVA

```java
static String describe(Object value) {
    return switch (value) {
        case null -> "null";
        case Point(int x, int y) when x == y -> "diagonal " + x;
        case Point(int x, int y) -> x + "," + y;
        default -> value.toString();
    };
}
```

Case labels are checked for dominance. An earlier unconditional supertype pattern can make a later subtype case unreachable. Guards refine a matching case. Exhaustiveness rules depend on the selector type; enum and sealed hierarchies are common interview examples.

Library and platform matrix

Feature	Release	Core idea	Boundary to remember
module system	9	reliable configuration and strong encapsulation	modules complement, not replace, packages/JARs
collection factories	9/10	`List.of`, `Set.of`, `Map.of`, `copyOf`	unmodifiable and null-rejecting, not necessarily a new object
standard HTTP client	11	synchronous/asynchronous HTTP/2-capable client	application still owns timeouts, body limits, retries, and executors
helpful NPE messages	14	more precise dereference context	diagnostic behavior, not a null-safety type system
strong encapsulation progression	9-17	restrict unsupported JDK internals	migration should remove reliance on internals, not add permanent opens
random generator interfaces	17	named/splittable/jumpable algorithms	choose by statistical/concurrency need, not habit
simple web server	18	minimal local HTTP serving API	not a full production service framework
UTF-8 default charset	18	standardized default charset	explicit charsets remain best at protocol/file boundaries
sequenced collections	21	uniform first/last/reversed encounter-order APIs	only meaningful for collections with a defined encounter order
virtual threads	21	cheap JVM-managed thread-per-task	do not pool to limit threads; bound external resources
generational ZGC	21	generational mode option for ZGC	collector flags/production maturity are release-specific

Sequenced collections

Java 21 adds `SequencedCollection`, `SequencedSet`, and `SequencedMap` to express defined encounter order and operations at both ends.

Representative methods:

JAVA

```
list.getFirst();
list.getLast();
list.addFirst(value);
list.reversed();

map.firstEntry();
map.lastEntry();
map.sequencedKeySet();
map.reversed();
```

Support and mutation behavior still depend on the concrete collection. A reversed view is generally a view, not an independent copy. An unmodifiable collection remains unmodifiable through its reversed view.

Virtual threads

Creation patterns:

JAVA

```java
Thread thread = Thread.ofVirtual().start(task);

try (var executor = java.util.concurrent.Executors.newVirtualThreadPerTaskExecutor()) {
    var future = executor.submit(task);
    future.get();
}
```

Virtual threads preserve the `Thread` programming model and are most compelling for large numbers of mostly blocking tasks. They improve scalability by allowing a virtual thread to unmount from a carrier during many blocking operations. They do not make CPU-bound work cheaper, change happens-before rules, or make shared mutable state safe.

Operational considerations:

- Replace thread-pool size as admission control with explicit semaphores, connection pools, rate limits, and bounded queues at constrained resources.
- Thread-local values multiplied across very many tasks can retain surprising state; keep them deliberate and small.
- Some blocking while holding a monitor or executing native/foreign code can pin a virtual thread to a carrier in Java 21. Diagnose with release-appropriate JFR/tooling rather than guessing.
- Preserve cancellation and structured lifecycle even when threads are inexpensive.

Structured concurrency and scoped values were preview APIs in Java 21 and are not used as final Java 21 APIs in this book.

API removals, encapsulation, and migration

Modernizing from Java 8/11 to 17/21 is more than changing a compiler level:

1. Inventory runtime distribution, build plugins, bytecode agents, annotation processors, JNI libraries, and reflective frameworks.
2. Run `jdeps` and tests to find internal JDK dependencies and removed modules.
3. Update dependencies before using broad `--add-opens` workarounds.
4. Check default charset, TLS/security defaults, GC defaults, container sizing, and logging.
5. Compile with `--release` and treat illegal reflective access or deprecation warnings as migration work.
6. Benchmark representative startup, steady-state, allocation, memory, and tail latency.
7. Roll out with canaries and explicit rollback criteria.

`--add-opens` can be a temporary compatibility bridge. It should name a specific need and have an owner/removal plan, because it weakens encapsulation and can hide unsupported coupling.

Preview and incubator discipline

Preview language/platform features require `--enable-preview` for compilation and execution with a matching release. Class files record preview usage. Incubator modules normally require `--add-modules` and can change or disappear.

Java 21's non-final features included:

Feature	Java 21 status	JEP
String Templates	preview	430
Unnamed Patterns and Variables	preview	443
Unnamed Classes and Instance Main Methods	preview	445
Foreign Function and Memory API	third preview	442
Scoped Values	preview	446
Structured Concurrency	preview	453
Vector API	sixth incubator	448

The iteration label matters: a later preview or incubator round is still not a final Java SE contract. Verify the exact JDK release before compiling examples or making a production compatibility promise.

Later-release delta: JDK 24 and JDK 25

This is context for interviews held after the Java 21 baseline. It does not change the source level of the book's runnable examples.

Later change	Delivered status	What a Java 21 answer should add
virtual-thread monitor pinning	JEP 491, delivered in JDK 24	Java 21 can pin a virtual thread during blocking inside `synchronized`; JDK 24 removes nearly all monitor-related pinning, while selected native and class-initialization cases can remain
scoped values	JEP 506, final in JDK 25	do not use the Java 21 preview API as a stable contract; describe the final JDK 25 API separately
module import declarations	JEP 511, final in JDK 25	this is source convenience and does not replace module readability, exports, or reliable configuration
compact source files and instance main methods	JEP 512, final in JDK 25	useful for small programs and learning; ordinary class-based source remains valid
flexible constructor bodies	JEP 513, final in JDK 25	statements may precede an explicit constructor invocation under the new rules; Java 21 retains the older first-statement restriction
compact object headers	JEP 519, delivered in JDK 25	object-header width is a VM/version/flag detail, so never present one layout as a Java guarantee
string templates	JEP 465 withdrawn	Java 21's preview syntax did not become a final feature; do not write production guidance as if STR were a current standard API
structured concurrency	fifth preview in JDK 25	the concept is useful, but preview signatures and lifecycle rules still require exact-release verification

The OpenJDK JDK 25 release page records the complete delivered feature set and its General Availability date. Long-term-support availability remains a distribution/vendor support decision even when a release is widely described as LTS.

This delta illustrates a durable interview habit: name the requested baseline first, then give a short later-release update. Do not silently answer a Java 21 question with JDK 25 behavior, and do not keep repeating a Java 21 limitation after the deployed runtime has removed it.

Interview answer pattern:

- State whether the feature is final, preview, or incubating in the named JDK.
- Describe the final Java 21 baseline separately from later evolution.
- Do not present a preview API signature as a stable production contract.
- Explain why an organization might experiment, and how it limits migration and support risk.

Feature selection checklist

Need	Relevant modern feature	Question before adoption
compact immutable-looking value carrier	record	are component values themselves safely immutable/copyable?
closed domain alternatives	sealed hierarchy	does the domain truly own and control all direct variants?
exhaustive type-based branching	pattern switch	are null, dominance, and future evolution handled?
deconstruct nested values	record pattern	is pattern logic clearer than behavior on the domain type?
readable embedded multi-line text	text block	are indentation, escaping, and untrusted input handled?
thread-per-request blocking service	virtual threads	which downstream resources still need explicit bounds?
uniform first/last/reverse access	sequenced collections	does the concrete collection define encounter order and supported mutation?
safer fixed collection boundary	factory/copy methods	is shallow immutability enough and are nulls prohibited?

Modern syntax is valuable when it sharpens the model. An SDE-2 answer also names compatibility, operability, lifecycle, and team-readability costs.

Part II - Practice and Handoff

Practice Ladder and Next Steps

TRY IT | Practice ladder

- Foundation: model classes and contracts
- Interview Core: equality, exceptions, and generics
- SDE-2 Follow-up: API evolution and type safety
- Challenge: modernize a design without semantic regressions

For every coding problem, state the input contract, select the numeric and collection types deliberately, write the invariant before the loop or recursion, test the smallest and largest valid inputs, and close with time and auxiliary-space complexity.

Navigation

Previous book: Study Step 18A - Advanced Java A: JVM and Execution

Series index

Next book: Study Step 18C - Advanced Java C: Collections, Streams, and I/O

TRY IT | Completion check

- [ ] I can recognize the core patterns without being told the data structure or algorithm.
- [ ] I can implement the representative Java templates from memory.
- [ ] I can explain why each implementation is correct.
- [ ] I can test boundaries, invalid inputs, overflow, and adversarial shapes.
- [ ] I can answer at least one SDE-2 follow-up involving trade-offs or production constraints.

Series Roadmap

Complete the study steps in order when rebuilding from fundamentals, or enter at the first step whose readiness checks expose a gap. The same public study codes appear on the website and every PDF cover. Click a title when your PDF viewer permits local sibling-file links.

STEP	FOCUSED LEARNING PATH
01	Java Foundations for Problem Solving
02	Time and Space Complexity for Java Interviews
03A	Number Systems and Math Foundations for DSA Interviews
03B	Number Systems Interview Patterns and Rapid Revision
04	Bit Manipulation in Java
05	Loop Mastery, Patterns, and Index Calculations
06	Arrays and Array Problem-Solving Patterns
07	Strings and String Problem-Solving Patterns
08	Hashing: Maps, Sets, Frequency, and Prefix State
09	Recursion and Backtracking in Java
10	Linked Lists: Pointer Reasoning and Mutation
11	Stacks, Queues, Deques, and Monotonic Patterns
12	Binary Search: Bounds, Answers, and Invariants
13	Trees, Binary Search Trees, and Tries
14	Heaps, Priority Queues, Selection, and Top-K
15	Graphs: Traversal, Ordering, Paths, and Union-Find
16	Greedy Algorithms: Recognition, Proofs, and Scheduling
17	Dynamic Programming: State, Transitions, and Optimization
18A	Advanced Java A: JVM and Execution
18B	Advanced Java B: Language, OOP, and Modern Java
18C	Advanced Java C: Collections, Streams, and I/O
18D	Advanced Java D: Concurrency and the Memory Model
18E	Advanced Java E: Performance, Diagnostics, and GC Incidents
18F	Advanced Java F: Design, Backend, Testing, and Security
18G	Advanced Java G: Question Bank, Study Plan, and Reference
18H	Advanced Java H: Spring Boot and REST APIs
18I	Advanced Java I: Persistence, SQL, JPA, and Caching
18J	Advanced Java J: Distributed Systems and System Design

Study Step 03 uses linked foundations (03A) and workbook (03B) books. Study Step 18 uses ten focused books (18A-18J), including a three-book backend specialist track. These suffixes keep every file focused while preserving one unambiguous route.

About the Author

Vinay Reddy Kalluri

Vinay Reddy Kalluri is a senior Java backend engineer, the creator and founding author of the Java SDE-2 Interview Preparation Series, and its Editor-in-Chief and Chief Auditor. His work spans high-throughput microservices, event-driven architecture, resilient APIs, large-scale data processing, cloud delivery, and production performance across healthcare and enterprise systems.

Editorial leadership

As Editor-in-Chief, Vinay owns the learning sequence, scope, voice, and publication decisions. As Chief Auditor, he owns the Java accuracy standard, validation evidence, attribution review, and release-readiness gate. Individual contributors retain visible credit for accepted original work through AUTHORS.md and the public Git history.

What distinguishes his approach is the combination of hands-on system design and operational ownership. He treats timeouts, retries, idempotency, dead-letter recovery, data consistency, SQL behavior, caching, and observability as part of the design rather than afterthoughts. He has also mentored engineers and led modernization, migration, and reliability work across distributed services.

Selected engineering and academic highlights

- More than eight years building Java and Spring Boot services for high-throughput healthcare and enterprise platforms.
- Designed Kafka-based event systems that have processed more than one million events per hour, with explicit idempotency, retry, recovery, and observability controls.
- M.S. in Computer Science from the University of Missouri-Kansas City (3.9/4.0) and M.S. in Software Engineering from VIT University (9.3/10), where he was a Gold Medalist.
- Independent builder of Work Visa Insights, an immigration-data intelligence platform, and Play Together, a published Android application.

Why this series exists

Vinay created this series to turn fragmented interview preparation into a deliberate progression: learn the fundamentals, run the examples, predict behavior, debug failures, explain trade-offs, and only then move into advanced engineering. The books reflect the same principle he applies to production systems: clarity before cleverness, explicit contracts, measurable behavior, and reliable foundations.

Connect: LinkedIn | GitHub

Copyright and Publishing Notes

Examples target Java 21 unless a section states otherwise. Validate release-specific APIs, JVM behavior, security, performance, licensing, and operational requirements before production use.

Series position: Study Step 18B of 18 - Part B of J. The local table of contents and PDF bookmarks navigate within this file; the roadmap and printed filenames navigate across the complete series.

Source provenance: curated from the independent Java SDE-2 master guide plus focused original material. Original master chapter numbers are labeled explicitly when retained in cross-references.